

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Embedded Systems (CO3053)

Assignment 2 Report

**Coin-Operated Washing
Machine Control Unit on the
WeAct STM32H750**

Advisor(s): Assoc. Prof. Phạm Hoàng Anh

Student(s):	Võ Hoàng Nguyên	2352844
	Nguyễn Quốc Thắng	2353122
	Ngô Nguyễn Thành Nhân	2352849

HO CHI MINH CITY, OCTOBER 2026



► **VIDEO DEMO — running on the real board**

Demonstration video of the coin-operated washing machine control unit running on the real board.

Watch on YouTube:

<https://youtube.com/shorts/ZFKOPDyJHqQ>

(Click the link, or scan the QR code with a phone.)



Contents

1	Introduction	7
1.1	Objectives of the assignment	7
1.2	The washing machine in short	7
1.3	Main functions	7
1.4	Scope of the work	8
2	System Requirements	8
2.1	Inputs	8
2.2	Outputs	10
2.3	Functional requirements	10
2.4	Non-functional requirements	11
2.5	Assumptions made by the team	11
3	System Analysis	13
3.1	Behaviour of the system	13
3.2	States	14
3.3	Transition conditions (informal)	14
4	Finite State Machine Design	15
4.1	Model	15
4.2	State diagram	15
4.3	State transition table	17
4.4	Output table (Moore outputs)	19
4.5	Design remarks	19



5	Hardware Design	20
5.1	Board	20
5.2	Mapping the specification onto the board	20
5.3	Pin mapping	21
5.4	Connection diagram	23
5.5	LCD	23
6	Software Architecture	24
6.1	Layered block diagram	24
6.2	Modules	25
6.3	Decoupling through callbacks	27
7	Timing Design	28
7.1	Timing parameters	28
7.2	One time base, no blocking delay	28
7.3	The 30-minute timer	29
7.4	Timer keeps running in PAUSED	29
7.5	LED blinking	29
7.6	Double STOP detection	30
8	Software Implementation	30
8.1	States, events and context	30
8.2	Main FSM: event dispatcher	31
8.3	Coin handling and RUN (no refund)	32
8.4	STOP handling	33
8.5	Timer update	34
8.6	LED update	34
8.7	Debounce logic	35
8.8	Main loop on the STM32	35
9	Flowcharts	37
9.1	Main program	37
9.2	Coin processing	38
9.3	RUN / PAUSE / STOP handling	39
10	Test Plan	40
10.1	Strategy	40



10.2 System test cases	40
10.3 Test environment	43
11 Experimental Results	43
11.1 Automated test results	43
11.2 Test scenarios on the board	44
11.3 LED indication in each state	46
11.4 LCD screens	46
11.5 Serial terminal log	47
11.6 Resource usage	47
12 Discussion	48
12.1 Problems met and how they were solved	48
12.2 Limitations	49
13 Conclusion	50
A Source code	52
A.1 FSM	52
A.2 Input and output engines	71
A.3 Board firmware	84
B Pin configuration	97
C Full test logs	98



List of Figures

4.1	State diagram – normal operation (single STOP presses, T05/T12, only arm the 1.5s window and are not drawn)	16
4.2	State diagram – fault handling (ERROR state)	16
5.1	Hardware connection diagram	23
5.2	LCD dashboard layout (frame rendered from the firmware’s own UI code, make preview)	24
6.1	Software architecture	25
7.1	Derivation of all timings from the 1 ms SysTick	29
7.2	STOP double-press window	30
9.1	Main program flowchart (non-blocking super-loop)	37
9.2	Coin processing in the FSM	38
9.3	Coin staging on the board’s single-key front panel	39
9.4	RUN / PAUSE / STOP handling (the 1 ms tick resets n when w expires) .	39
11.1	Photos of the real board: LCD dashboard in each state	46
11.2	LCD dashboard in each state (frames rendered from the firmware UI code)	47

List of Tables

2.1	Logical inputs of the control unit	9
2.2	Outputs of the control unit	10
2.3	Functional requirements	10
2.4	Design assumptions	12
3.1	States of the control unit	14
4.1	Complete state transition table	17
4.2	Outputs per state	19
5.1	Main board resources used	20
5.2	K1 gesture map (front panel with coin staging, <code>wm_two_button.c</code>)	21
5.3	Pin mapping	22
6.1	Software modules	25
7.1	Timing parameters (<code>washing_machine_config.h</code> , <code>hal_button_gesture.h</code>)	28
10.1	System test cases	40
10.2	Test environment	43
11.1	Automated test results	44
11.2	Test scenarios and expected behaviour on the WeAct STM32H750	45



11.3 Firmware size (arm-none-eabi-size build/h750/washer.elf)	47
B.1 Register-level pin configuration	98



1 Introduction

1.1 Objectives of the assignment

The goal of this assignment is to design and implement the *control unit* of a coin-operated washing machine, following the full embedded-system workflow: requirements → analysis → finite-state-machine (FSM) design → implementation on a real microcontroller → verification. The report is organised to answer four questions:

1. **What must the system do?** (Sections 2–3)
2. **How is it designed?** (Sections 4–7)
3. **How is it implemented?** (Sections 8–9)
4. **How do we prove it works?** (Sections 10–11)

1.2 The washing machine in short

The machine is a self-service, coin-operated washer such as those found in laundromats and dormitories. The assignment statement given in class is:

1. The machine can be controlled by three buttons (STOP, RUN, PAUSE). 2. The machine has two LEDs including Red LED (RLED) and Blue one (BLED): RLED is on when the machine is on standby (i.e. it is available to serve); RLED is blinking if the machine has errors; BLED is on when the machine is ready to execute; BLED is blinking when the machine is running (do washing). 3. The washing machine is only able to execute once at least 50 cents is added. 4. The machine only accepts 10-cent, 20-cent, 50-cent coins. 5. When there is enough money (BLED is on), the machine will execute if the RUN button is pressed. Then, a 30-mins timer will be activated and the money will be cleared without returning the redundancies (if any). 6. When the machine is running (BLED is blinking), it will be paused as the PAUSE button is pressed but the timer is still counting down. The machine will re-execute when the RUN button is pressed again. 7. The executing machine will be automatically terminated after 30 minutes (washing cycle) or when the STOP button is pressed twice (force to stop).

1.3 Main functions

- **Coin acceptance:** 10¢, 20¢ and 50¢ coins are accumulated; at least 50¢ is required to run; surplus money is never refunded on RUN.



- **RUN / PAUSE / STOP control:** RUN starts or resumes washing, PAUSE suspends washing while the timer keeps counting, STOP pressed twice forces the machine to stop.
- **30-minute cycle timer:** a non-blocking countdown that ends the cycle automatically.
- **LED indication:** two LEDs (RLED, BLED) with ON / OFF / 1 Hz / 2 Hz patterns that encode the current state of the machine.

1.4 Scope of the work

The work focuses on the **control logic**. Concretely, the deliverables are:

- a portable, hardware-independent FSM written in C99 (`src/fsm/`);
- non-blocking input/output engines: debounce, coin-pulse decoder, LED blinker, single-button gesture decoder (`src/hal/`, `src/app/`);
- a bare-metal firmware for the **WeAct Studio STM32H750VBT6** core board with a 0.96" ST7735 LCD dashboard and a UART console (`board/weact_h750/`);
- an automated host test suite (94 test cases in five suites) and an interactive simulator.

Out of scope. The real drum motor, water valve, drain pump, door lock and coin acceptor are *not* driven by real power electronics. Their commands are produced by the FSM through a callback interface and are shown as indicators on the LCD and in the UART log (Section 5). The hardware-abstraction layer is designed so that real relays or a real pulse-output coin acceptor can be connected later without touching the FSM.

2 System Requirements

2.1 Inputs

Table 2.1 lists the logical inputs of the controller. “Logical” means the input as seen by the FSM; Section 5 explains how each one is produced on the board.

Table 2.1: Logical inputs of the control unit

Input	FSM event	Meaning
STOP	WM_EVT_BTN_STOP	One press of STOP. Two presses within 1.5 s = force stop.
RUN	WM_EVT_BTN_RUN	Start a cycle (from READY) or resume (from PAUSED).
PAUSE	WM_EVT_BTN_PAUSE	Suspend washing; the timer keeps counting.
Coin 10¢	WM_EVT_COIN_10	A 10-cent coin was accepted.
Coin 20¢	WM_EVT_COIN_20	A 20-cent coin was accepted.
Coin 50¢	WM_EVT_COIN_50	A 50-cent coin was accepted.
Error signal	WM_EVT_FAULT_OCCURRED	A safety fault (door open, water timeout, motor overcurrent).
Error cleared	WM_EVT_FAULT_CLEARED	The fault has been removed / acknowledged.
1 ms tick	wm_fsm_tick_1ms()	Time base for the STOP double-press window.
1 s tick	WM_EVT_TIMER_TICK_1S	Time base for the 30-minute countdown.

2.2 Outputs

Table 2.2: Outputs of the control unit

Output	Values	Meaning
RLED	OFF / ON / BLINK 2 Hz	ON = standby (available); blinking = error.
BLED	OFF / ON / BLINK 1 Hz	ON = ready (or paused); blinking = washing.
Motor	OFF / AGITATE / SPIN	Washing enable. AGITATE in the first 5/6 of the cycle, SPIN in the last 1/6.
Drain pump	on / off	On during the final spin.
Water valve	open / closed	Kept closed in this version (no fill sensor).
Door lock	locked / unlocked	Locked while a cycle is active, released on stop/error.
Coin return	amount (¢)	Only when a deposit is cancelled <i>before</i> RUN (see A6).

2.3 Functional requirements

Table 2.3: Functional requirements

ID	Requirement
FR-01	The machine accepts only 10¢, 20¢ and 50¢ coins and accumulates their value.
FR-02	The machine can execute only when the accumulated balance is ≥ 50 ¢; BLED is then ON.
FR-03	Surplus money is not refunded: when RUN starts the cycle, the whole balance is cleared to 0.
FR-04	Pressing RUN in READY starts the washing cycle and activates a 30-minute timer.
FR-05	The cycle lasts 30 minutes (1800 s).



ID	Requirement
FR-06	Pressing PAUSE while running stops washing, but the timer keeps counting down.
FR-07	Pressing RUN while paused resumes washing with the <i>remaining</i> time (timer not reset).
FR-08	Pressing STOP twice forces the machine to stop and return to standby. A single press has no effect.
FR-09	When the timer reaches 0 (running <i>or</i> paused) the machine returns to standby automatically.
FR-10	LED indication: RLED ON in standby, RLED blinking on error, BLED ON when ready, BLED blinking when running.
FR-11	When an error occurs, all actuators are switched off immediately and RLED blinks.

2.4 Non-functional requirements

- **NFR-01 Non-blocking.** No busy-wait delay in the control loop; every timing (debounce, blink, double press, 30-minute timer) is derived from a 1 ms tick.
- **NFR-02 Deterministic.** The FSM is a pure function of (state, event); the same event sequence always gives the same outputs, so it can be tested on a PC.
- **NFR-03 Portable.** The FSM does not touch registers; outputs go through a call-back table so the same code runs on the PC, the simulator and the STM32.
- **NFR-04 Robust.** Defensive checks for NULL pointers, counter overflow and unknown enum values (MISRA-C style).

2.5 Assumptions made by the team

The statement leaves several points open. The team fixed them as follows; every assumption is covered by at least one automated test.

Table 2.4: Design assumptions

ID	Assumption
A1	Double STOP window = 1.5 s. The second STOP must arrive within 1500ms of the first; otherwise the first press is discarded. A single press never stops the machine.
A2	Buttons ignored in STANDBY. RUN, PAUSE and STOP have no effect when no money has been inserted. RUN with less than 50¢ is also ignored.
A3	Coins after READY. Coins inserted in READY are still accepted (balance grows) and are cleared on RUN (no refund). Coins are rejected in RUNNING, PAUSED and ERROR.
A4	BLED in PAUSED is solid ON. The statement does not define it; solid ON means “ready to continue” and is clearly different from blinking (running).
A5	COLLECTING state. $0 < \text{balance} < 50¢$ is a separate state. For the user it looks like STANDBY (RLED ON, BLED OFF), but STOP is meaningful there (A6).
A6	Cancel before RUN returns the deposit. $\text{STOP} \times 2$ in COLLECTING/READY returns the deposit through the optional <code>return_coins()</code> callback. The no-refund rule applies only to RUN (FR-03).
A7	Error source. The statement mentions errors but not their source. Faults are modelled as a bitmask (door open, water timeout, motor overcurrent) and are <i>injected</i> on the board for the demo.
A8	Error recovery keeps context. Clearing a fault restores the deposit; a cycle that was running or paused resumes in PAUSED and its timer kept counting during the fault (same rule as PAUSE). Build option <code>WM_ERROR_RESUMES_CYCLE=0</code> gives the simple “always return to STANDBY” behaviour.
A9	Wash profile. The motor agitates during the first 5/6 of the cycle and spins with the drain pump on during the last 1/6 (1500s + 300s).



ID	Assumption
A10	Demo time scale. On the board the cycle is shortened to 30 s so the whole cycle can be shown live; <code>make h750 FULL_CYCLE=1</code> builds the real 1800 s version. The PC tests always use 1800 s.

3 System Analysis

3.1 Behaviour of the system

From the customer's point of view a washing session has three phases:

1. **Payment.** The customer inserts coins one by one. The machine only needs to remember the balance and compare it with 50¢. Nothing mechanical happens.
2. **Washing.** After RUN, the machine owns a *time budget* of 30 minutes. The customer may pause and resume, but pausing does not buy extra time: the budget is consumed by wall-clock time, not by motor time. This is the most important observation of the analysis: *the timer belongs to the session, not to the motor*.
3. **End.** The session ends when the budget is used up, or when the customer deliberately confirms a stop by pressing STOP twice. A single accidental touch must not cancel a paid cycle.

Errors (e.g. the door is opened) can happen in any phase and must override everything else: the actuators are switched off first, then the user is informed by a blinking RLED.

Two different kinds of information therefore have to be kept:

- a small number of **qualitative modes** (paying, ready, washing, paused, error) that decide how each button is interpreted and what the LEDs show $\Rightarrow$ the *states* of an FSM;
- **quantitative data** (balance, remaining seconds, STOP window) that change inside a mode $\Rightarrow$ *extended state variables* kept in a context structure, used in guard conditions.

This leads to an *extended finite-state machine*: 6 states plus 5 context variables:



<code>coin_balance_cents</code>	accumulated deposit
<code>remaining_cycle_sec</code>	countdown of the washing cycle
<code>stop_press_count</code>	STOP presses inside the current window (0 or 1)
<code>stop_window_timer_ms</code>	remaining time of the 1.5s double-press window
<code>active_error_flags</code>	bitmask of active faults

3.2 States

Table 3.1: States of the control unit

State	LEDs	Meaning
STANDBY	RLED ON, BLED OFF	Idle and available to serve; balance = 0; all actuators off, door unlocked. Initial state after power-on.
COLLECTING	RLED ON, BLED OFF	Some money inserted but $< 50\text{¢}$. Still “available” for the user, so the LEDs are the same as STANDBY.
READY	RLED OFF, BLED ON	Balance $\geq 50\text{¢}$; waiting for RUN. More coins can still be inserted.
RUNNING	RLED OFF, BLED 1 Hz	Washing: door locked, motor agitating or spinning, timer counting down.
PAUSED	RLED OFF, BLED ON	Motor, pump and valve off; door stays locked; timer still counting down.
ERROR	RLED 2 Hz, BLED OFF	Safety fault: every actuator off and door unlocked; all buttons and coins locked out until the fault is cleared.

3.3 Transition conditions (informal)

- STANDBY/COLLECTING $\rightarrow$ COLLECTING or READY: a coin is inserted; the target depends on whether the new balance reaches 50¢.
- READY $\rightarrow$ RUNNING: RUN is pressed. The balance is cleared, the timer is loaded with 1800 s.
- RUNNING $\leftrightarrow$ PAUSED: PAUSE / RUN. The timer is *not* reset when resuming.
- RUNNING/PAUSED $\rightarrow$ STANDBY: the timer reaches 0, or STOP is pressed twice within 1.5 s.



- COLLECTING/READY $\rightarrow$ STANDBY: STOP pressed twice (customer cancels; deposit returned).
- any state $\rightarrow$ ERROR: a fault is detected.
- ERROR $\rightarrow$ previous context: the fault is cleared (PAUSED if a cycle was in progress and time remains, otherwise STANDBY/COLLECTING/READY according to the balance).

Events that are not listed for a state (for example PAUSE in READY, or a coin in RUNNING) are **ignored**: the FSM stays in the same state and produces no output. This is checked systematically by test TC-27 (event-acceptance matrix).

4 Finite State Machine Design

4.1 Model

The controller is an **extended FSM** with 6 states and the context variables

$b = \text{coin_balance_cents}$, $t = \text{remaining_cycle_sec}$, $n = \text{stop_press_count}$, $w = \text{stop_window_timer_ms}$, $T = \text{cycle length (1800 s)}$.

LED outputs depend only on the state (Moore outputs, Table 4.2); the updates of b , t , n and the coin return are attached to transitions (Mealy actions, Table 4.1). Notation on the diagrams: *event* [*guard*] / *action*; COIN(v) stands for any of the three coin events with value $v \in \{10, 20, 50\}$.

4.2 State diagram

Figure 4.1 shows normal operation; Figure 4.2 shows how the ERROR state is entered from, and returns to, the rest of the machine. Transition numbers Txx refer to Table 4.1.

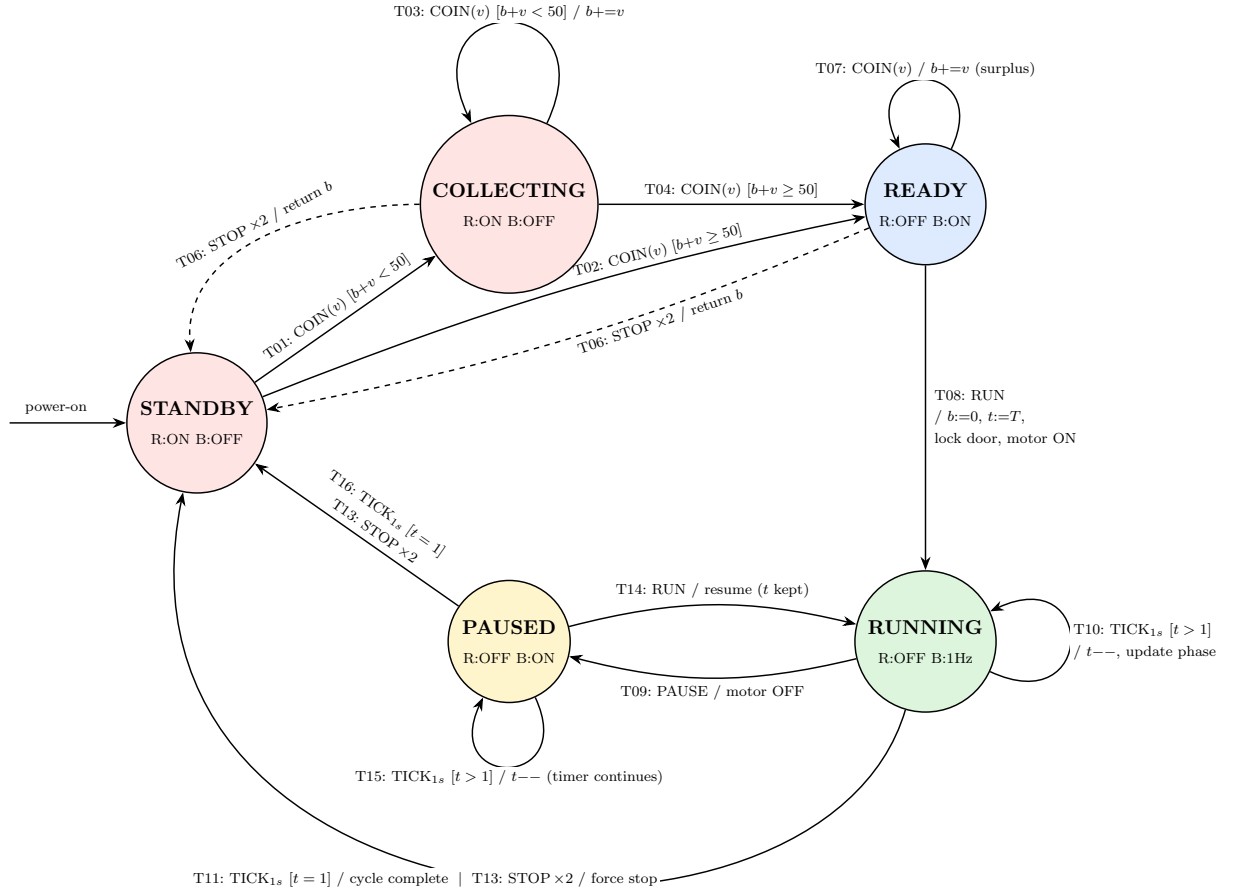


Figure 4.1: State diagram – normal operation (single STOP presses, T05/T12, only arm the 1.5 s window and are not drawn)

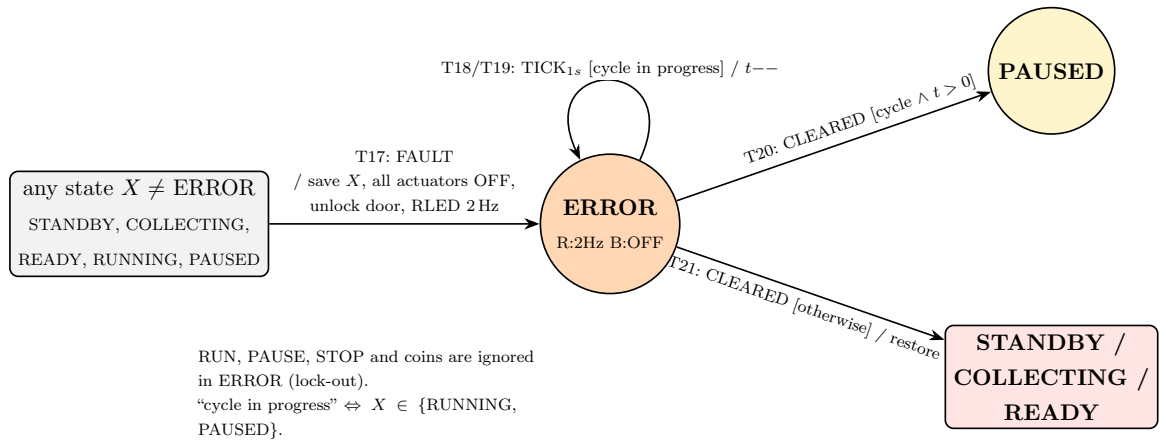


Figure 4.2: State diagram – fault handling (ERROR state)



4.3 State transition table

Table 4.1: Complete state transition table

ID	Current state	Event	Guard	Next state	Action / output
T01	STANDBY	COIN(v)	$b + v < 50$	COLLECTING	$b += v$; RLED ON, BLED OFF
T02	STANDBY	COIN(v)	$b + v \geq 50$	READY	$b += v$; RLED OFF, BLED ON
T03	COLLECTING	COIN(v)	$b + v < 50$	COLLECTING	$b += v$; reset STOP counter
T04	COLLECTING	COIN(v)	$b + v \geq 50$	READY	$b += v$; RLED OFF, BLED ON
T05	COLLECTING, STOP READY		$n = 0$	(same)	$n := 1$; $w := 1500$ ms
T06	COLLECTING, STOP READY		$n = 1$ ($w > 0$)	STANDBY	return_coins (b); $b := 0$; RLED ON, BLED OFF
T07	READY	COIN(v)	–	READY	$b += v$ (surplus kept, never refunded on RUN)
T08	READY	RUN	–	RUNNING	$b := 0$ (no refund); $t := T$; RLED OFF; BLED 1 Hz; lock door; motor AGITATE
T09	RUNNING	PAUSE	–	PAUSED	motor, valve, pump OFF; BLED ON; door stays locked; t not frozen
T10	RUNNING	TICK _{1s}	$t > 1$	RUNNING	$t -= 1$; motor AGITATE if $t > T/6$, else SPIN + drain pump
T11	RUNNING	TICK _{1s}	$t = 1$	STANDBY	$t := 0$; on_cycle_complete (); all OFF; unlock; RLED ON



ID	Current state	Event	Guard	Next state	Action / output
T12	RUNNING, PAUSED	STOP	$n = 0$	(same)	$n := 1$; $w := 1500$ ms; washing/timer unaffected
T13	RUNNING, PAUSED	STOP	$n = 1$ $(w > 0)$	STANDBY	force stop: all OFF; unlock door; RLED ON, BLED OFF
T14	PAUSED	RUN	–	RUNNING	resume: BLED 1 Hz; actuators by phase; t <i>not</i> reloaded
T15	PAUSED	TICK _{1s}	$t > 1$	PAUSED	$t -= 1$ (timer continues while paused)
T16	PAUSED	TICK _{1s}	$t = 1$	STANDBY	$t := 0$; on_cycle_complete(); RLED ON
T17	any $\neq$ ERROR	FAULT	–	ERROR	save state; motor, valve, pump OFF; unlock; BLED OFF; RLED 2 Hz
T18	ERROR	TICK _{1s}	cycle $\wedge$ $t > 1$	ERROR	$t -= 1$
T19	ERROR	TICK _{1s}	cycle $\wedge$ $t = 1$	ERROR	$t := 0$; cycle finished (saved state := STANDBY)
T20	ERROR	CLEARED	cycle $\wedge$ $t > 0$	PAUSED	flags := 0; BLED ON; user presses RUN to continue
T21	ERROR	CLEARED	otherwise	STANDBY / COLLECT- ING / READY	flags := 0; state chosen from b (deposit kept)
T22	any	TICK _{1ms}	$w > 0$	(same)	$w -= 1$; if $w = 0$ then $n := 0$ (single press discarded)

Ignored events. Every (state, event) pair that is not in Table 4.1 is ignored, e.g. RUN/PAUSE/STOP in STANDBY, RUN in COLLECTING (not enough money),



PAUSE in READY, coins in RUNNING/PAUSED/ERROR, all buttons in ERROR. `wm_fsm_dispatch_event()` returns `false` for these and the outputs do not change.

4.4 Output table (Moore outputs)

Table 4.2: Outputs per state

State	RLED	BLED	Motor	Pump	Door	Timer
STANDBY	ON	OFF	OFF	off	unlocked	idle ($t = 0$)
COLLECTING	ON	OFF	OFF	off	unlocked	idle
READY	OFF	ON	OFF	off	unlocked	idle
RUNNING	OFF	1 Hz	AGITATE / SPIN	on in SPIN	locked	counting
PAUSED	OFF	ON	OFF	off	locked	counting
ERROR	2 Hz	OFF	OFF	off	unlocked	counting if a cycle was active

4.5 Design remarks

- **Why the timer is a variable and not a state.** 1800 states would be absurd; a counter with guards $t > 1$ / $t = 1$ keeps the diagram small and lets RUNNING and PAUSED share the same countdown code (T10/T15), which guarantees FR-06.
- **Why ERROR has priority.** The FAULT event is checked *before* the state switch in `wm_fsm_dispatch_event()`, so no state can “forget” to handle it.
- **Why STOP needs a counter.** The double press is a small sub-machine ($n \in \{0, 1\}$ plus the window w) embedded in every state that accepts STOP. It is reset whenever the machine enters STANDBY, COLLECTING, RUNNING, PAUSED or ERROR, so a first press made before RUN or PAUSE can never combine with a press made after it.



5 Hardware Design

5.1 Board

The target is the **WeAct Studio MiniSTM32H7xx core board** with an **STM32H750VBT6** (Arm Cortex-M7, 128 KB internal flash, 1 MB RAM)^[1;2;3]. The board was chosen because it already carries the parts needed for the demo: a user key, a user LED, a 0.96" LCD connector and USB-C for DFU flashing.

Table 5.1: Main board resources used

Resource	Use in this project
STM32H750VBT6, 400 MHz	Runs the firmware (HSE 25 MHz $\rightarrow$ PLL1 $\rightarrow$ 400 MHz CPU, 200 MHz AHB, 100 MHz APB; HSI fallback if the crystal fails)
SysTick	1 kHz system tick (time base of every timing in the system)
Key K1 (PC13)	The only software-readable button: all coins and RUN/PAUSE/STOP via gestures
LED PE3	Physical status LED, mirrors RLED $\vee$ BLED
0.96" ST7735 LCD	160 $\times$ 80 dashboard: state, timer, balance, RLED/BLED and actuator indicators
USART1 (PA9/PA10)	115 200 8N1 status log and keyboard control from a PC
USB-C / BOOT0 / NRST	Power and DFU flashing
SWD (PA13/PA14)	Optional ST-Link flashing/debugging

5.2 Mapping the specification onto the board

The specification asks for three buttons, three coin inputs and two LEDs. The WeAct board has three physical keys, but only one of them can be read by software:

- **K1** on PC13 is a normal GPIO input (active HIGH, R8 10 k Ω + C4 100 nF on the board).
- **NRST** resets the MCU and cannot be read.



- **BOOT0** is a dedicated pin on the STM32H7 (not a GPIO) and is sampled only at reset^[1].

The team therefore implemented two complementary input paths:

1. **K1 gestures** (no extra hardware): a gesture decoder turns one key into click, double-click and two-stage hold, which are mapped onto coins and RUN/PAUSE/STOP depending on the state (Table 5.2).
2. **Optional external buttons** on PA0 (K0: coin shortcut / RUN-PAUSE) and PA1 (STOP), wired to GND and using internal pull-ups, plus the UART console keys.

The two LEDs are shown as coloured RLED/BLED boxes on the LCD (driven by the real blinker engine), and the onboard PE3 LED shows RLED $\vee$ BLED so the 1 Hz and 2 Hz blink patterns are also visible on a physical LED. The motor, pump and door-lock commands are shown as AGIT/SPIN, PUMP and LOCK indicators.

Table 5.2: K1 gesture map (front panel with coin staging, `wm_two_button.c`)

K1 gesture	STANDBY	READY	RUNNING	PAUSED	ERROR
Click	+10¢	RUN	PAUSE	RUN (resume)	clear fault
Double-click	+20¢	fault	fault	fault	clear fault
Hold 0.6 s and release	+50¢	STOP #1	STOP #1	STOP #1	–
Keep holding to 1.2 s	confirm coins	STOP #2	STOP #2	STOP #2	clear fault

“fault” = inject a door-open fault (`WM_FAULT_DOOR_OPEN`) to demonstrate the ERROR state. In STANDBY the coins are first *staged* and shown in the top-right corner of the LCD; holding K1 to 1.2 s confirms them: with $\geq 50¢$ the FSM receives the payment and goes to READY, with $< 50¢$ the panel shows NOT ENOUGH and the staged coins are returned. This staging step is a user-interface layer *on top of* the FSM; the FSM and its tests are unchanged. Because K1 hold-to-1.2 s produces the first STOP at 0.6 s and the second at 1.2 s, the double-STOP rule of the specification (two presses within 1.5 s) is reproduced exactly.

5.3 Pin mapping



Table 5.3: Pin mapping

Function	Pin	Mode	Notes
K1: coins + RUN/PAUSE/STOP	PC13	Input, pull-down	Active HIGH, onboard RC filter
K0 (ext.): coin / RUN-PAUSE	PA0	Input, pull-up	Optional, button to GND, active LOW
STOP (ext.)	PA1	Input, pull-up	Optional, button to GND, active LOW
Coin 10/20/50¢	–	–	No coin acceptor; coins from K1/K0 gestures or UART keys. The driver <code>hal_coin_pulse</code> (1/2/5 pulses) is ready for a real acceptor.
Status LED (RLED ∨ BLED)	PE3	Output push-pull	Active HIGH
RLED, BLED	LCD	–	Drawn on the LCD from the blinker outputs
LCD SCK	PE12	AF5 (SPI4)	12.5 MHz, TX only
LCD MOSI	PE14	AF5 (SPI4)	
LCD CS	PE11	Output	
LCD DC	PE13	Output	Data/command select
LCD backlight	PE10	AF1 (TIM1_CH2N)	10 kHz PWM
LCD RST	NRST	–	Wired to the MCU reset on the board
UART TX / RX	PA9 / PA10	AF7 (USART1)	115 200 8N1, 3.3 V USB-TTL
SWDIO / SWCLK	PA13 / PA14	SWD	Optional ST-Link

5.4 Connection diagram

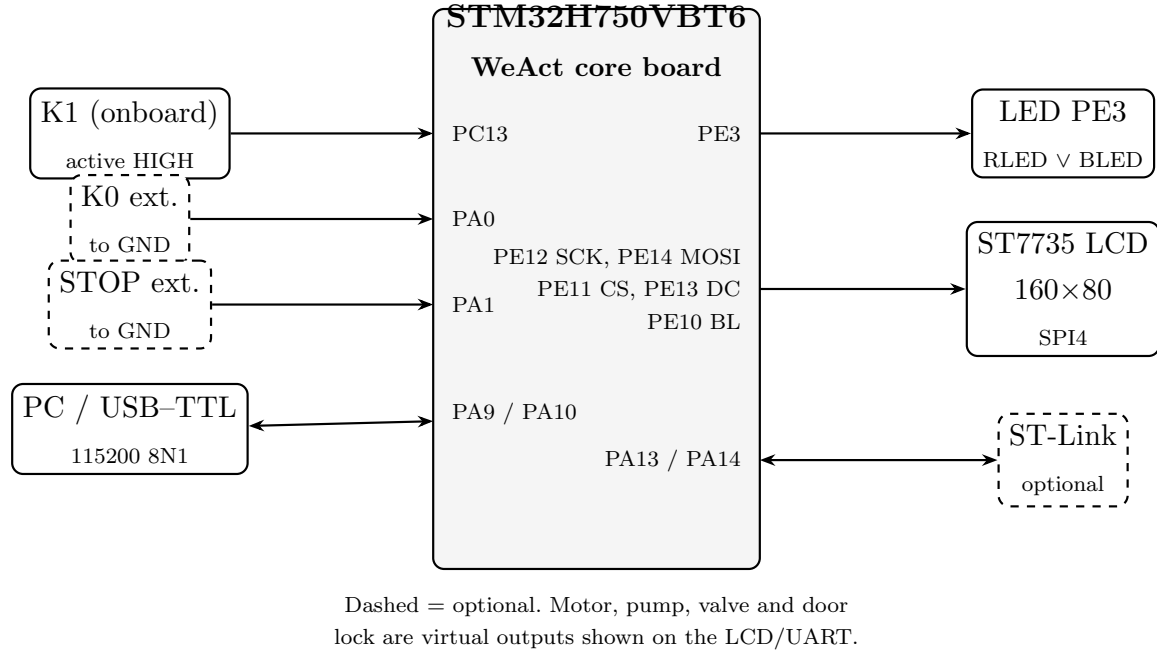


Figure 5.1: Hardware connection diagram

5.5 LCD

- **Model:** 0.96" IPS module, ST7735 controller^[4], 160×80 pixels, RGB565, plugged into the board's LCD connector. Two panel variants exist (HannStar/BOE); the BOE one is selected with `make h750 LCD_PANEL=BOE`.
- **Interface:** SPI4 in transmit-only master mode at 12.5 MHz; a full frame (25.6 KB) is rendered into a RAM frame buffer and pushed only when the displayed data change.
- **Purpose in the demo:** the board has only one LED and no actuators, so the LCD makes the whole FSM visible: state name (colour-coded), remaining time, wash phase (WASH/SPIN), balance, RLED/BLED, motor/pump/lock indicators, a progress bar and a hint line for the next K1 action.



Figure 5.2: LCD dashboard layout (frame rendered from the firmware's own UI code, make preview)

6 Software Architecture

6.1 Layered block diagram

The software is split into four layers. Only the lowest layer knows the STM32 registers; the FSM does not know whether it runs on the board, in the PC simulator or in a unit test.

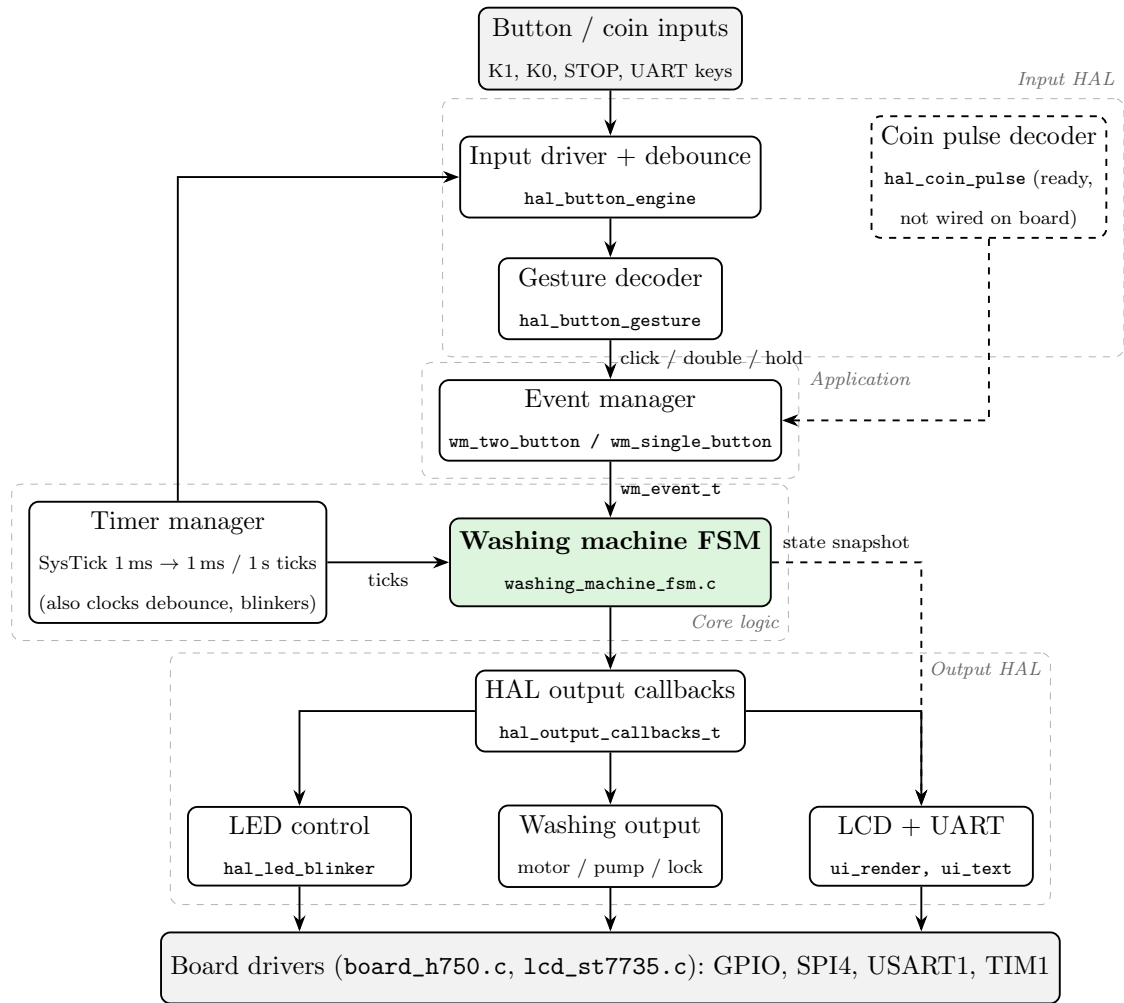


Figure 6.1: Software architecture

6.2 Modules

Table 6.1: Software modules

Module (file)	Responsibility
GPIO / input driver <code>board_h750.c</code>	Configures PC13 (pull-down), PA0/PA1 (pull-up), PE3 output; returns the raw pin level. No logic.
Debounce <code>hal_button_engine.c</code>	Per-button integrator: the raw level must differ from the stable level for 30 consecutive ms before the stable level changes. Produces latched <i>pressed/released</i> edges. Works for active-LOW and active-HIGH buttons.



Module (file)	Responsibility
Gesture decoder <code>hal_button_gesture.c</code>	Small FSM (IDLE, DOWN1, WAIT, DOWN2, HELD) that turns the debounced K1 level into CLICK, DOUBLE, HOLD1 (0.6s), HOLD1_END and HOLD2 (1.2s).
Coin processing <code>wm_two_button.c</code> , <code>hal_button_engine.c</code>	Front panel: stages coins from gestures and confirms them into the FSM. For a real coin acceptor, the <code>hal_coin_pulse</code> decoder counts pulse trains (1 = 10¢, 2 = 20¢, 5 = 50¢; 150 ms silence ends a coin) and buffers up to four coins in a FIFO.
Event manager <code>wm_single_button.c</code>	Translates a gesture into an FSM event <i>according to the current state</i> (e.g. click = RUN in READY, PAUSE in RUNNING).
FSM <code>washing_machine_fsm.c</code>	The core: state, balance, timer, STOP window, fault flags. Single entry point <code>wm_fsm_dispatch_event()</code> ; outputs only through callbacks. No dependency on hardware.
Timer manager <code>main_h750.c</code>	SysTick ISR increments a millisecond counter; the main loop catches up one millisecond at a time and derives the 1 ms and 1 s ticks (Section 7).
LED control <code>hal_led_blinker.c</code>	Non-blocking blinker per LED: OFF, ON, 1 Hz (500/500 ms), 2 Hz (250/250 ms). The FSM sets the <i>mode</i> ; the blinker produces the waveform. Also contains the actuator interlock check <code>hal_actuator_is_safe()</code> .
Washing output (callbacks in <code>main_h750.c</code>)	Receives motor/pump/valve/door-lock commands. On this board they update the LCD indicators; on a real machine they would drive relays.
LCD module <code>lcd_st7735.c</code> , <code>gfx.c</code> , <code>ui_render.c</code>	ST7735 init over SPI4, a small 2-D library on a RAM frame buffer, and the dashboard renderer. Redraws only when the visible data change.
UART console <code>ui_text.c</code>	One status line per change (no <code>printf</code> , no heap) and single-key commands for remote control.



6.3 Decoupling through callbacks

The FSM receives a table of function pointers at initialisation:

Listing 1: Output interface of the FSM (`hal_interfaces.h`)

```
1 typedef struct {  
2     void (*set_rled)(hal_led_state_t state);  
3     void (*set_bled)(hal_led_state_t state);  
4     void (*set_motor)(hal_motor_state_t state);  
5     void (*set_water_valve)(bool open);  
6     void (*set_drain_pump)(bool on);  
7     void (*set_door_lock)(bool locked);  
8     void (*on_cycle_complete)(void);  
9     void (*return_coins)(uint32_t cents); /* optional */  
10 } hal_output_callbacks_t;
```

The same FSM object file is linked into five programs: the unit tests (with a recording mock), the HAL tests, the interactive CLI simulator, the LCD preview tool and the STM32 firmware. Any callback may be `NULL`; the FSM checks before calling it.



7 Timing Design

7.1 Timing parameters

Table 7.1: Timing parameters (`washing_machine_config.h`, `hal_button_gesture.h`)

Parameter	Value	Source / reason
System tick (SysTick)	1 ms	SysTick_Config(400 MHz / 1000)
Debounce time	30 ms	Longer than typical contact bounce (5–20 ms), still feels instant
STOP double-press window	1500 ms	Assumption A1
Gesture: double-click gap	300 ms	Max release→press gap; a click is confirmed 300 ms after release
Gesture: HOLD1 / HOLD2	600 / 1200 ms	HOLD2 – HOLD1 = 600 ms < 1500 ms STOP window
BLED blink (RUNNING)	1 Hz	500 ms ON / 500 ms OFF
RLED blink (ERROR)	2 Hz	250 ms ON / 250 ms OFF (faster = more urgent)
Washing cycle	1800 s	FR-05 (30 s in the default board demo build, A10)
Spin phase	last $T/6$	300 s of 1800 s (5 s in the demo build)
Coin pulse silence timeout	150 ms	End of a pulse train (real coin acceptor)
UART status line	on change	At most once per second while running

7.2 One time base, no blocking delay

`HAL_Delay()`-style waiting would freeze the controller: during a 500 ms delay for blinking, a button press or a coin could be lost and the 30-minute timer would drift. Instead, every timing in the system is a **counter advanced by a 1 ms tick**:

- The SysTick interrupt does only one thing: `g_millis++`. It is short, cannot block and shares no other data with the main loop, so no critical section is needed.
- The main loop compares `board_millis()` with the last processed millisecond and calls `process_1ms()` *once per elapsed millisecond*. If the loop was busy (e.g. pushing

a 25 KB frame to the LCD takes a few ms), it catches up the missing milliseconds in order, so no tick is lost and all timers stay exact.

- `process_1ms()` advances, in a fixed order: the STOP window, both LED blinkers, the debouncers, the gesture decoders, and a millisecond accumulator that produces the 1 s tick every 1000 calls.

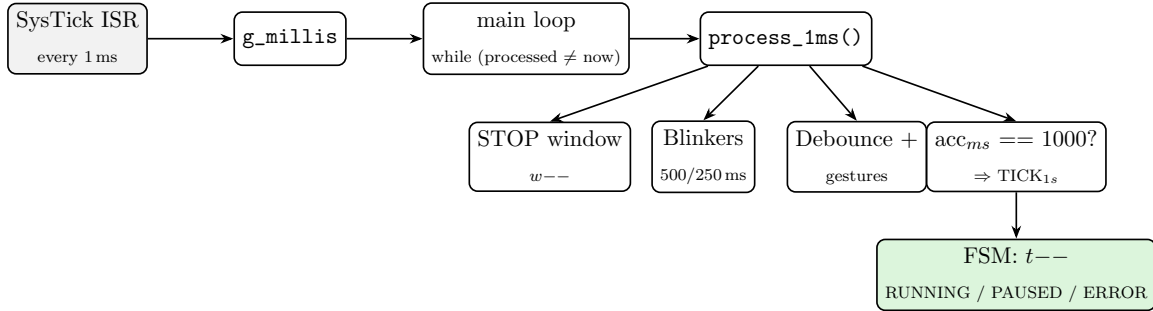


Figure 7.1: Derivation of all timings from the 1 ms SysTick

7.3 The 30-minute timer

The timer is the context variable `remaining_cycle_sec`. RUN in READY loads it with `cycle_duration_setting` (1800s). Each 1 s tick decrements it by one; when it goes from 1 to 0 the FSM calls `on_cycle_complete()` and enters STANDBY. Because the 1 s tick is generated by counting exactly 1000 SysTick periods, the accuracy of the timer equals the accuracy of the 25 MHz crystal (typically ± 20 ppm, i.e. < 0.04 s over 30 min).

7.4 Timer keeps running in PAUSED

The same handler `handle_1s_timer_tick()` is called from both RUNNING and PAUSED (transitions T10 and T15). The only difference is that RUNNING also updates the actuators for the current phase, while PAUSED leaves them off. Therefore pausing *cannot* stop the timer, and a cycle that is paused until $t = 0$ ends in STANDBY exactly as a running one (T16, test TC-08).

7.5 LED blinking

The FSM only sets a *mode* (`HAL_LED_BLINK_1HZ, ...`). The blinker keeps a phase accumulator and toggles the output every half-period:

$$f_{\text{BLED}} = \frac{1}{2 \times 500 \text{ ms}} = 1 \text{ Hz}, \quad f_{\text{RLED}} = \frac{1}{2 \times 250 \text{ ms}} = 2 \text{ Hz}.$$

Changing the mode restarts the phase with the LED ON, so the first blink after entering RUNNING is always a full 500 ms ON. If a large time step is passed ($\text{delta_ms} \gg \text{period}$), the number of toggles is computed with a modulo so the ON/OFF parity is preserved (no drift, test HAL-03).

7.6 Double STOP detection

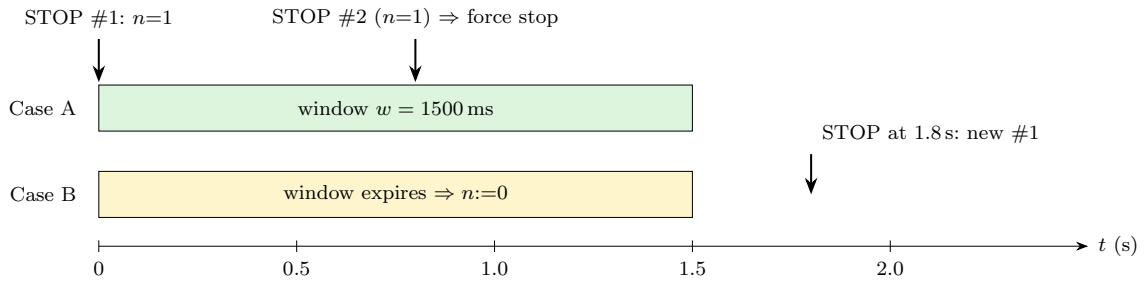


Figure 7.2: STOP double-press window

The first STOP sets $n := 1$ and $w := 1500$. Every millisecond w is decremented; when it reaches 0, n is reset and the first press is forgotten. A STOP that arrives while $n = 1$ is the confirmed second press and forces the stop. The boundary is tested at 1499 ms (stop) and 1501 ms (no stop) in TC-22. Washing and the timer are not affected by the first press.

8 Software Implementation

Only the key fragments are shown here; the full sources are listed in Appendix A. The code is C99, compiled with `-Wall -Wextra -Werror -pedantic` for both the PC and the Cortex-M7 (`arm-none-eabi-gcc -mcpu=cortex-m7 -Os`). The firmware uses no HAL library, no `printf` and no heap: 9.3 KB of flash in total.

8.1 States, events and context

Listing 2: State and event enumerations (`washing_machine_fsm.h`)

```
1 typedef enum {
2     WM_STATE_STANDBY = 0,      /* RLED=ON,  BLED=OFF */
3     WM_STATE_COLLECTING,      /* 0 < deposit < 50c; RLED=ON, BLED=OFF */
4     WM_STATE_READY,          /* deposit >= 50c;   RLED=OFF, BLED=ON */
5     WM_STATE_RUNNING,        /* BLED blink 1 Hz, timer ticking */
6     WM_STATE_PAUSED,         /* actuators stopped, timer STILL ticking */
```

```
7   WM_STATE_ERROR          /* RLED blink 2 Hz, actuators stopped */
8 } wm_state_t;
9
10 typedef enum {
11     WM_EVT_NONE = 0, WM_EVT_COIN_10, WM_EVT_COIN_20, WM_EVT_COIN_50,
12     WM_EVT_BTN_RUN, WM_EVT_BTN_PAUSE, WM_EVT_BTN_STOP,
13     WM_EVT_TIMER_TICK_1S, WM_EVT_TIMER_TICK_1MS,
14     WM_EVT_FAULT_OCCURRED, WM_EVT_FAULT_CLEARED
15 } wm_event_t;
16
17 typedef struct {
18     wm_state_t state;
19     uint32_t coin_balance_cents;      /* b */
20     uint32_t remaining_cycle_sec;     /* t */
21     uint32_t stop_press_count;        /* n */
22     uint32_t stop_window_timer_ms;    /* w */
23     uint32_t active_error_flags;
24     hal_output_callbacks_t callbacks;
25     uint32_t cycle_duration_setting;  /* T = 1800 s */
26     wm_state_t state_before_error;
27 } wm_context_t;
```

All the data of the machine live in one `wm_context_t`; there are no global variables in the FSM, so several independent machines could run side by side (the tests create a fresh one for each case).

8.2 Main FSM: event dispatcher

The FSM is a `switch` on the current state with a nested `switch` on the event. Each transition calls an `enter_xxx()` function that sets the new state and performs the entry actions (Moore outputs), so the LED rules are written once per state and cannot be forgotten on one of the incoming transitions.

Listing 3: Event dispatcher (abridged, `washing_machine_fsm.c`)

```
1 bool wm_fsm_dispatch_event(wm_context_t *ctx, wm_event_t event) {
2     if (!ctx) return false;
3
4     /* Global emergency transition: has priority over every state (T17) */
5     if (event == WM_EVT_FAULT_OCCURRED) {
6         if (ctx->active_error_flags == WM_FAULT_NONE)
7             ctx->active_error_flags = WM_FAULT_DOOR_OPEN;
8         if (ctx->state != WM_STATE_ERROR) { enter_error(ctx); return true; }
```

```
9     return false;
10 }
11 if (event == WM_EVT_TIMER_TICK_1MS) { wm_fsm_tick_1ms(ctx); return true; }
12
13 switch (ctx->state) {
14 case WM_STATE_READY:
15     switch (event) {
16     case WM_EVT_COIN_10: return handle_coin_deposit(ctx, 10);
17     case WM_EVT_COIN_20: return handle_coin_deposit(ctx, 20);
18     case WM_EVT_COIN_50: return handle_coin_deposit(ctx, 50);
19     case WM_EVT_BTN_RUN: enter_running(ctx, false); return true;    /* T08 */
20     case WM_EVT_BTN_STOP: return handle_stop_button(ctx);
21     default:              return false;                            /* ignored */
22     }
23 case WM_STATE_RUNNING:
24     switch (event) {
25     case WM_EVT_BTN_PAUSE: enter_paused(ctx); return true;        /* T09 */
26     case WM_EVT_BTN_STOP: return handle_stop_button(ctx);
27     case WM_EVT_TIMER_TICK_1S: return handle_1s_timer_tick(ctx);  /* T10/T11 */
28     default:              return false;
29     }
30 case WM_STATE_PAUSED:
31     switch (event) {
32     case WM_EVT_BTN_RUN: enter_running(ctx, true); return true; /* T14
33 resume */
34     case WM_EVT_BTN_STOP: return handle_stop_button(ctx);
35     case WM_EVT_TIMER_TICK_1S: return handle_1s_timer_tick(ctx);  /* T15: timer
36 continues */
37     default:              return false;
38     }
39     /* ... STANDBY, COLLECTING, ERROR handled the same way ... */
40 default:
41     return false;
42 }
```

8.3 Coin handling and RUN (no refund)

Listing 4: Coin accumulation and start of the cycle

```
1 static bool handle_coin_deposit(wm_context_t *ctx, uint32_t amount) {
2     if (ctx->state == WM_STATE_STANDBY || ctx->state == WM_STATE_COLLECTING) {
```

```
3     if (ctx->coin_balance_cents <= (UINT32_MAX - amount))    /* overflow guard */
4         ctx->coin_balance_cents += amount;
5     if (ctx->coin_balance_cents >= WM_COIN_THRESHOLD_CENTS)
6         enter_ready(ctx);    /* T02/T04: RLED OFF, BLED ON */
7     else
8         enter_collecting(ctx);    /* T01/T03 */
9     return true;
10 } else if (ctx->state == WM_STATE_READY) {
11     if (ctx->coin_balance_cents <= (UINT32_MAX - amount))
12         ctx->coin_balance_cents += amount; /* T07: surplus accepted */
13     return true;
14 }
15 return false;    /* rejected in RUNNING/PAUSED/ERROR */
16 }
17
18 static void enter_running(wm_context_t *ctx, bool is_resuming) {
19     ctx->state = WM_STATE_RUNNING;
20     ctx->stop_press_count = 0;
21     ctx->stop_window_timer_ms = 0;
22     if (!is_resuming) {
23         ctx->coin_balance_cents = 0;    /* clear money, no refund (FR-03) */
24         ctx->remaining_cycle_sec = ctx->cycle_duration_setting;    /* 1800 s */
25     }
26     CALL_HAL(&ctx->callbacks, set_rled, HAL_LED_OFF);
27     CALL_HAL(&ctx->callbacks, set_bled, HAL_LED_BLINK_1HZ);
28     update_running_actuators(ctx);    /* lock door, AGITATE or SPIN */
29 }
```

8.4 STOP handling

Listing 5: Double-press STOP

```
1 static bool handle_stop_button(wm_context_t *ctx) {
2     if (ctx->state == WM_STATE_STANDBY) return false;
3     if (ctx->stop_press_count == 0) {    /* T05/T12: first press */
4         ctx->stop_press_count = 1;
5         ctx->stop_window_timer_ms = WM_DOUBLE_PRESS_WINDOW_MS;    /* 1500 ms */
6         return true;
7     }
8     /* T06/T13: second press inside the window -> force stop */
9     ctx->stop_press_count = 0;
10    ctx->stop_window_timer_ms = 0;
```

```
11     if ((ctx->state == WM_STATE_READY || ctx->state == WM_STATE_COLLECTING) &&
12         ctx->coin_balance_cents > 0)
13         CALL_HAL(&ctx->callbacks, return_coins, ctx->coin_balance_cents);
14     ctx->coin_balance_cents = 0;
15     enter_standby(ctx);           /* all actuators OFF, RLED ON */
16     return true;
17 }
18
19 void wm_fsm_tick_1ms(wm_context_t *ctx) {    /* T22 */
20     if (ctx->stop_window_timer_ms > 0) {
21         ctx->stop_window_timer_ms--;
22         if (ctx->stop_window_timer_ms == 0)
23             ctx->stop_press_count = 0;      /* window expired: forget 1st press */
24     }
25 }
```

8.5 Timer update

Listing 6: 1-second tick, shared by RUNNING and PAUSED

```
1 static bool handle_1s_timer_tick(wm_context_t *ctx) {
2     if (ctx->remaining_cycle_sec > 1) {
3         ctx->remaining_cycle_sec--;          /* T10 / T15 */
4         if (ctx->state == WM_STATE_RUNNING)
5             update_running_actuators(ctx);  /* AGITATE -> SPIN at T/6 */
6         return true;
7     } else if (ctx->remaining_cycle_sec == 1) {
8         ctx->remaining_cycle_sec = 0;        /* T11 / T16 */
9         CALL_HAL_VOID(&ctx->callbacks, on_cycle_complete);
10        enter_standby(ctx);
11        return true;
12    }
13    return false;
14 }
```

8.6 LED update

Listing 7: Non-blocking blinker (hal_led_blinker.c)

```
1 void hal_led_blinker_tick_ms(hal_led_blinker_t *b, uint32_t delta_ms) {
2     if (!b || b->half_period_ms == 0U) return; /* OFF or ON: nothing to do */
```

```
3   if (delta_ms > (UINT32_MAX - b->phase_timer_ms))    /* overflow guard */
4       delta_ms %= (2U * b->half_period_ms);
5   b->phase_timer_ms += delta_ms;
6   if (b->phase_timer_ms >= b->half_period_ms) {
7       uint32_t toggles = b->phase_timer_ms / b->half_period_ms;
8       if ((toggles & 1U) != 0U)
9           b->output_level = !b->output_level;          /* 500 ms (1 Hz) or 250 ms
10              (2 Hz) */
11       b->phase_timer_ms %= b->half_period_ms;
12   }
```

8.7 Debounce logic

Listing 8: Integrating debouncer (hal_button_engine.c)

```
1 void hal_button_update(hal_button_t *btn, bool raw_pin_high, uint32_t delta_ms) {
2     bool pressed = btn->active_low ? !raw_pin_high : raw_pin_high;
3     if (pressed != btn->debounced_state) {
4         btn->debounce_counter += delta_ms;              /* level differs: count */
5         if (btn->debounce_counter >= HAL_DEBOUNCE_DEFAULT_MS) { /* 30 ms stable */
6             btn->debounced_state = pressed;
7             btn->debounce_counter = 0;
8             if (pressed) btn->pressed_event = true;      /* latched edge */
9             else         btn->released_event = true;
10        }
11    } else {
12        btn->debounce_counter = 0;                        /* bounce: restart */
13    }
14 }
```

Any glitch shorter than 30 ms resets the counter, so a bouncing contact produces exactly one edge. The edge is latched and consumed once by `hal_button_was_pressed()`.

8.8 Main loop on the STM32

Listing 9: Super-loop and 1 ms housekeeping (main_h750.c, abridged)

```
1 void SysTick_Handler(void) { g_millis++; }           /* the only ISR work */
2
3 static void process_1ms(uint32_t *sec_acc_ms) {
4     wm_fsm_tick_1ms(&g_fsm);                          /* STOP window */
5 }
```



```
5     hal_led_blinker_tick_ms(&g_rled, 1U);           /* LED waveforms */
6     hal_led_blinker_tick_ms(&g_bled, 1U);
7     hal_button_update(&g_key, board_key_pressed(), 1U);           /* debounce */
8     hal_gesture_t g = hal_gesture_update(&g_gesture, hal_button_is_pressed(&g_key),
9     1U);
10    if (g != HAL_GESTURE_NONE) k1_event(g);           /* gesture -> FSM event */
11    /* ... external KO / STOP buttons ... */
12    if (++(*sec_acc_ms) >= 1000U) {                   /* 1 s tick */
13        *sec_acc_ms = 0U;
14        wm_fsm_tick_1s(&g_fsm);
15    }
16
17    int main(void) {
18        board_init();                                 /* clocks, GPIO, SysTick 1 kHz
19        */
20        wm_fsm_init(&g_fsm, &cbs);                   /* -> STANDBY, RLED ON */
21        lcd_init();
22        for (;;) {
23            uint32_t now = board_millis();
24            while (processed != now) { processed++; process_1ms(&sec_acc_ms); }
25            /* UART keys, status log, PE3 LED = RLED | BLED */
26            build_view(&view);
27            if (!ui_view_equal(&view, &shown)) { ui_render(&view); lcd_flush(); shown =
28            view; }
29        }
```

9 Flowcharts

9.1 Main program

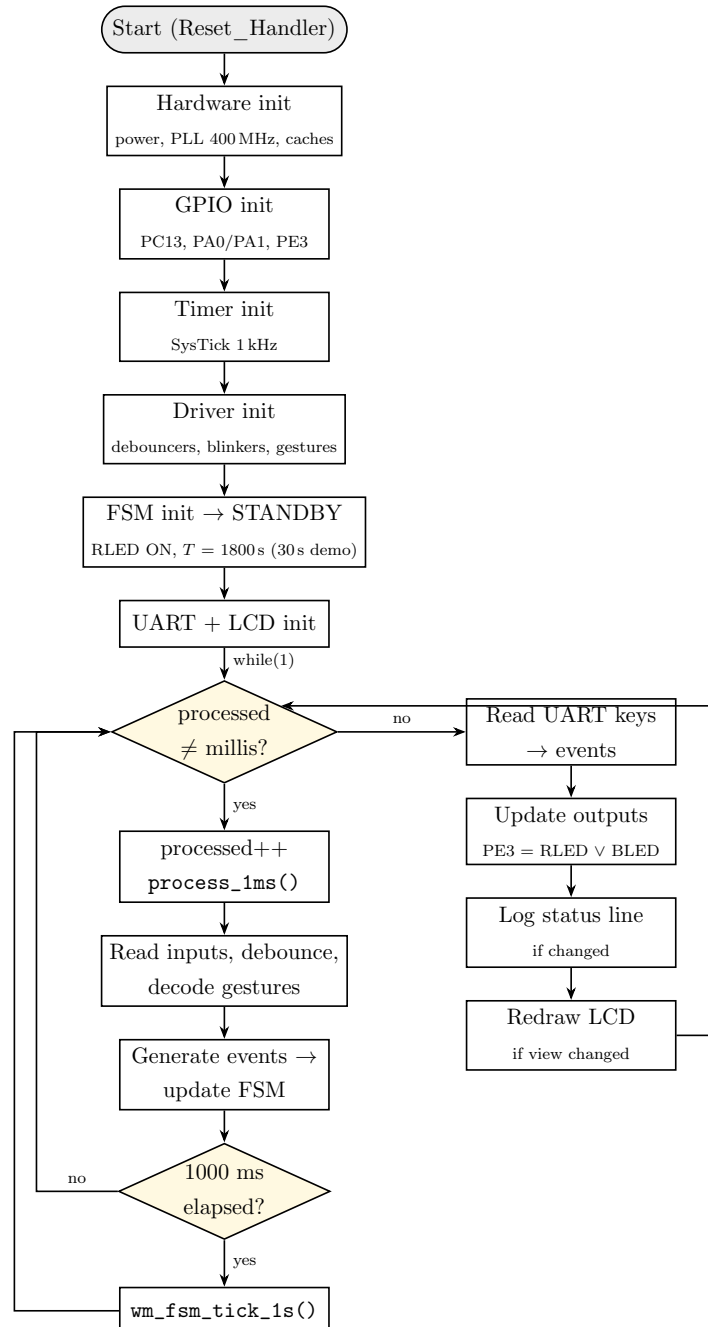


Figure 9.1: Main program flowchart (non-blocking super-loop)

9.2 Coin processing

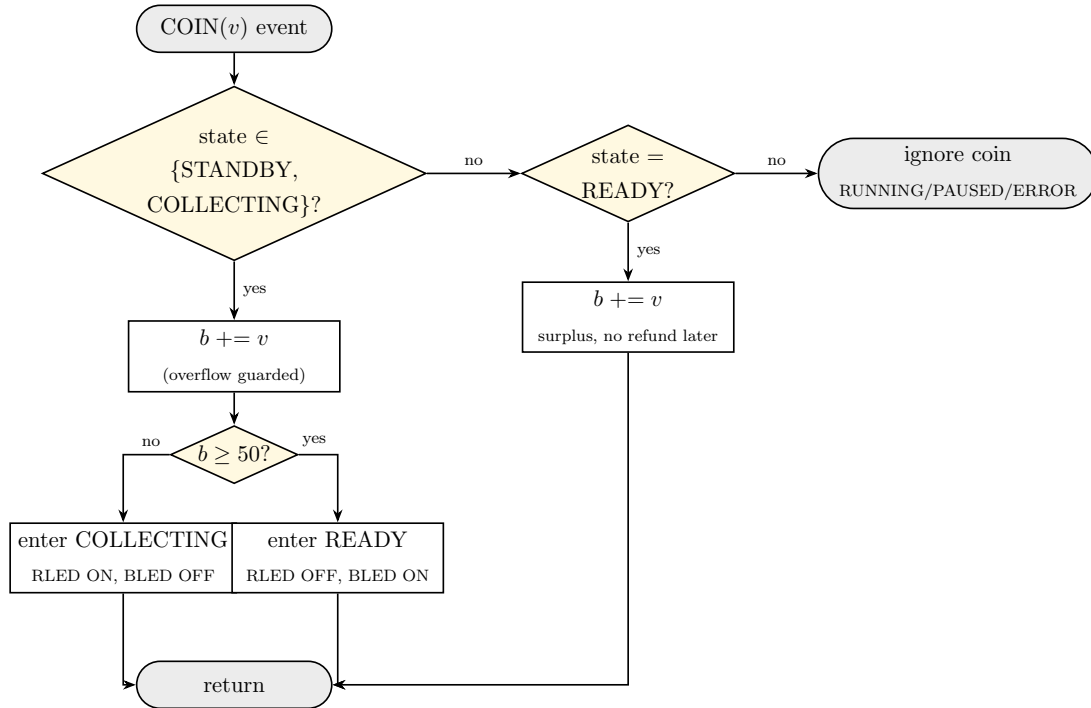


Figure 9.2: Coin processing in the FSM

On the board, coins come from K1 gestures and are first *staged* by the front panel (Figure 9.3); a real coin acceptor would instead feed `hal_coin_pulse`, which emits `COIN(v)` events directly.

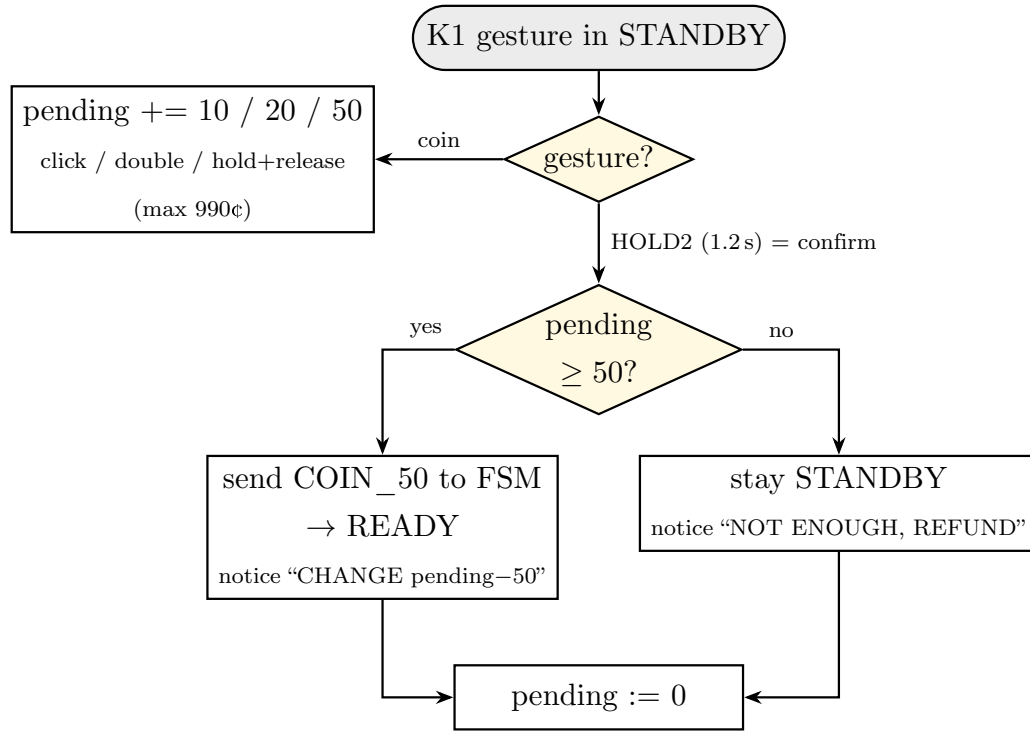


Figure 9.3: Coin staging on the board's single-key front panel

9.3 RUN / PAUSE / STOP handling

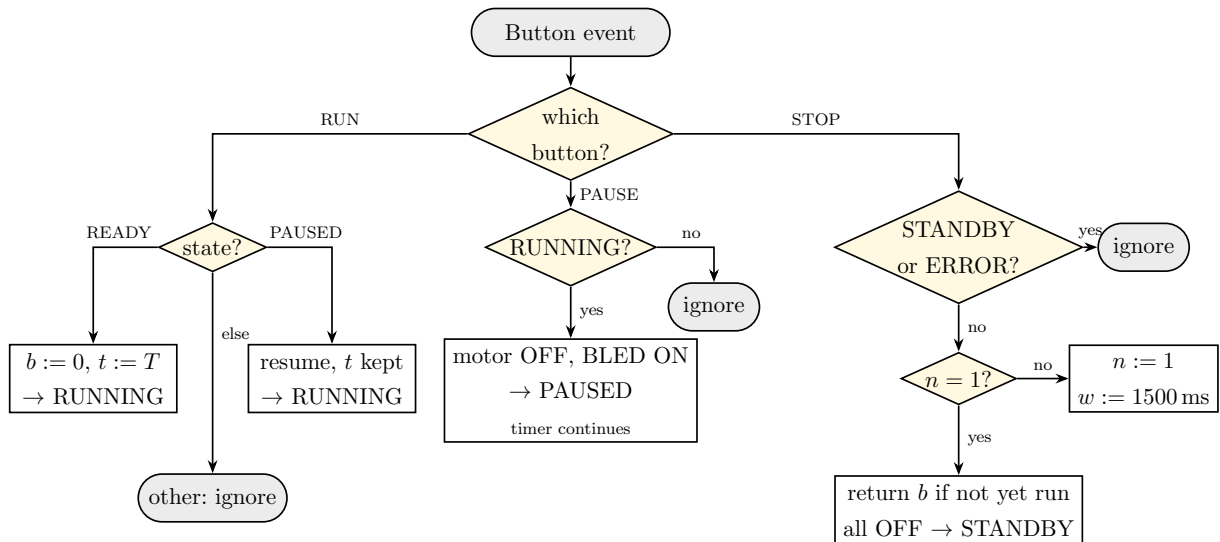


Figure 9.4: RUN / PAUSE / STOP handling (the 1 ms tick resets n when w expires)



10 Test Plan

10.1 Strategy

The test cases were written *before* running them and are executed at two levels:

1. **Automated host tests (PC).** The exact same FSM, debounce, blinker and gesture code is compiled with `gcc` and driven by event sequences. A mock HAL records every output (LED modes, motor, pump, door lock, returned coins). Time is simulated, so a 30-minute cycle is tested in milliseconds and boundary times (e.g. 1499ms vs 1501ms) are exact. Command: `make test test_hal test_gesture test_two_button test_ui_text`.
2. **Hardware tests (board).** The firmware is flashed on the WeAct STM32H750 and each scenario is performed with K1 (or the UART keys). The LCD, the PE3 LED and the UART log are observed and recorded (board photos and the demo video, Section 11).

Requirement coverage: every functional requirement FR-01...FR-11 and every assumption A1...A10 is checked by at least one test case below; every transition T01...T22 is exercised by the automated suite.

10.2 System test cases

Table 10.1: System test cases

ID	Scenario	Input / steps (board)	Expected result	Req. / auto test
TC01	Start machine	Power on / press NRST	STANDBY; RLED ON, BLED OFF; balance 000C; motor OFF, door unlocked	FR-10 / TC-01, TC-31
TC02	Insert 10¢	COIN10 (K1 click)	Balance = 10¢; RLED stays ON; BLED OFF (COLLECTING / staged)	FR-01 / TC-01, TC-31



ID	Scenario	Input / steps (board)	Expected result	Req. / auto test
TC03	Reach 50¢	20 + 20 + 10¢ (K1 double, double, click; then hold 1.2s to confirm)	READY; RLED OFF, BLED ON	FR-02 / TC-02, TWO-BTN
TC04	RUN with 30¢	10 + 20¢, then RUN	Machine does not start; stays COLLECTING (board: “NOT ENOUGH, REFUND 030C”, stays STANDBY)	FR-02, A2 / TC-05, TWO-BTN
TC05	RUN with 50¢	50¢, RUN (K1 click in READY)	RUNNING; timer = 30:00 (demo 00:30); BLED 1 Hz; balance 0; door locked; motor AGITATE	FR-04, FR-05 / TC-04
TC06	Pause	PAUSE while running (K1 click)	PAUSED; motor OFF; BLED ON; timer keeps counting down	FR-06 / TC-06, TC-07
TC07	Resume	RUN while paused (K1 click)	RUNNING; BLED 1 Hz; timer continues from the current value (not reset)	FR-07 / TC-06, TC-14
TC08	Timer expires	Let the cycle run to 0	STANDBY; RLED ON; motor OFF; door unlocked	FR-09 / TC-12, TC-21
TC09	Timer expires while paused	PAUSE and wait until 0	STANDBY (paused machine does not keep the time)	FR-06, FR-09 / TC-08, TC-29
TC10	STOP once	One STOP while running (K1 hold 0.6s, release)	Keeps running; nothing changes	FR-08 / TC-09, TC-19, TC-23



ID	Scenario	Input / steps (board)	Expected result	Req. / auto test
TC11	STOP twice	Two STOPs within 1.5 s (K1 hold to 1.2 s)	Force stop → STANDBY, RLED ON, all OFF	FR-08, A1 / TC-10, TC-11, TC-22
TC12	STOP twice, too slow	Two STOPs 1.501 s apart	Not stopped; the second press counts as a new first press	A1 / TC-22, TC-19
TC13	Insert 70¢ then RUN	50 + 20¢, RUN	Starts; balance cleared to 0; no refund	FR-03 / TC-03, TC-04
TC14	Coins during washing	COIN while RUNNING / PAUSED	Coin ignored; balance stays 0	A3 / TC-15
TC15	Cancel before RUN	READY with 60¢, STOP ×2	STANDBY; 60¢ returned ("CANCELLED, RETURN 060C")	A6 / TC-18, TC-32
TC16	Error	Fault while RUNNING (K1 double-click)	ERROR; RLED 2 Hz; BLED OFF; motor/pump OFF; door unlocked; buttons and coins locked out	FR-11 / TC-13, TC-16, TC-17
TC17	Error recovery	Clear fault (K1 click)	PAUSED, timer kept counting during fault; RUN resumes	A8 / TC-33, TC-34
TC18	Wash phases	Run a full cycle	AGITATE for first 5/6, then SPIN + PUMP for last 1/6	A9 / TC-26, TC-28
TC19	LED frequencies	Observe PE3 for 10 s in RUNNING and in ERROR	≈10 blinks (1 Hz) and ≈20 blinks (2 Hz)	FR-10 / HAL-03
TC20	Button bounce	Press K1 quickly many times	One event per press; no spurious double events	NFR-01 / HAL-01, gesture tests



Note on TC04 on the board. With the single-key front panel the coins are staged and confirmed by a 1.2s hold (Section 5); confirming with $< 50\text{¢}$ is the board equivalent of pressing RUN without enough money. The FSM behaviour itself (RUN ignored in COLLECTING) is verified by automated test TC-05.

10.3 Test environment

Table 10.2: Test environment

Host	Linux, gcc -std=c99 -Wall -Wextra -Werror -pedantic -O2
Target	WeAct STM32H750VBT6, arm-none-eabi-gcc -mcpu=cortex-m7 -Os, flashed by USB-DFU
Observation	0.96" LCD, PE3 LED, USART1 log at 115 200 baud (picocom), phone camera
Firmware build	Demo build (cycle = 30s) for scenarios TC05–TC18; FULL_CYCLE=1 build for one full 30-minute run

11 Experimental Results

11.1 Automated test results

All five host suites were built with `-Werror` and run; every test passed (full logs in Appendix C).



Table 11.1: Automated test results

Suite (make target)	What it verifies	Tests	Passed	Result
<code>test</code>	FSM: all transitions, guards, LEDs, timer, STOP window, faults, robustness	34	34	PASS
<code>test_hal</code>	Debounce, coin pulse decoder + FIFO, 1/2 Hz blinker, actuator interlock	6	6	PASS
<code>test_gesture</code>	K1 click / double / hold decoding and mapping to FSM events	32	32	PASS
<code>test_two_button</code>	Coin staging, confirm, change/refund notices, K0 shortcut	18	18	PASS
<code>test_ui_text</code>	UART status line formatting and truncation safety	4	4	PASS
Total		94	94	PASS

Listing 10: Excerpt of `make test` output

```

1  [PASS] TC-04: Execution & Zero Refund (70¢ cleared to 0¢, 30-min timer active)
2  [PASS] TC-05: Premature RUN Attempt (Ignored when balance < 50¢)
3  [PASS] TC-06: Normal Pause and Resume (Actuators safely suspended and resumed)
4  [PASS] TC-07: Persistent Timer in Pause (Timer ticked down from 1600s to 1300s while paused)
5  [PASS] TC-08: Pause Timeout Termination (Timer expiring in PAUSED resets to STANDBY)
6  [PASS] TC-09: Single STOP Rejection (Single press does not force stop)
7  [PASS] TC-10: Force Stop on Double Press (2 presses within 300ms forces termination)
8  [PASS] TC-22: Boundary Double-Stop Timing (Exact 1499ms hit vs 1501ms expiration verified)
9  =====
10 ALL 34 TESTS PASSED SUCCESSFULLY! (100% Specification & Transition Coverage)
11 =====

```

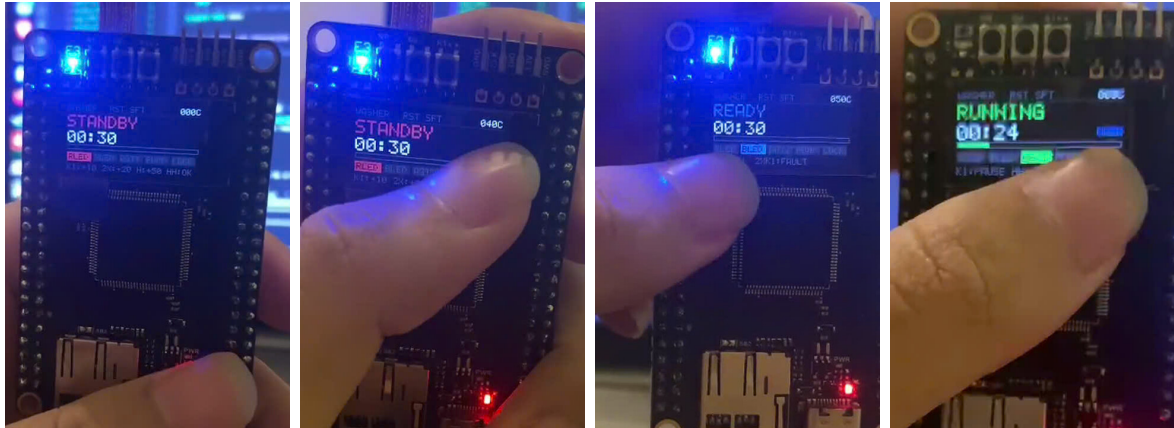
11.2 Test scenarios on the board

Table 11.2 lists the expected behaviour of each scenario of the test plan. Every scenario is checked by the automated suites above; the main states on the real board are shown in Figure 11.1 and in the demo video.

Table 11.2: Test scenarios and expected behaviour on the WeAct STM32H750

Test	Expected	Automated
TC01	STANDBY, RLED ON	PASS
TC02	Balance 010C, RLED ON	PASS
TC03	READY, BLED ON	PASS
TC04	Does not start (NOT ENOUGH)	PASS
TC05	RUNNING, timer starts, BLED 1 Hz	PASS
TC06	PAUSED, motor OFF, timer continues	PASS
TC07	RUNNING again, timer not reset	PASS
TC08	STANDBY at 00:00	PASS
TC09	STANDBY at 00:00 while paused	PASS
TC10	Keeps running	PASS
TC11	Force stop → STANDBY	PASS
TC12	Not stopped	PASS
TC13	Starts, balance 000C	PASS
TC14	Coin ignored	PASS
TC15	STANDBY, “RETURN 060C”	PASS
TC16	ERROR, RLED 2 Hz, all OFF	PASS
TC17	PAUSED after clear, RUN resumes	PASS
TC18	WASH then SPIN + PUMP	PASS
TC19	10 blinks / 10 s; 20 blinks / 10 s	PASS
TC20	One event per press	PASS

11.3 LED indication in each state



(a) STANDBY, 0¢ (b) coins staged, 40¢ (c) READY, 50¢ (d) RUNNING
RLED ON RLED ON BLED ON BLED 1 Hz

Figure 11.1: Photos of the real board: LCD dashboard in each state

11.4 LCD screens

Figure 11.2 shows the dashboard for each state. These frames are rendered on the PC by `make preview`, which runs the *same* FSM and `ui_render.c` as the firmware and dumps the frame buffer; they are pixel-identical to what the firmware sends to the panel. Photos of the real panel are in Figure 11.1.

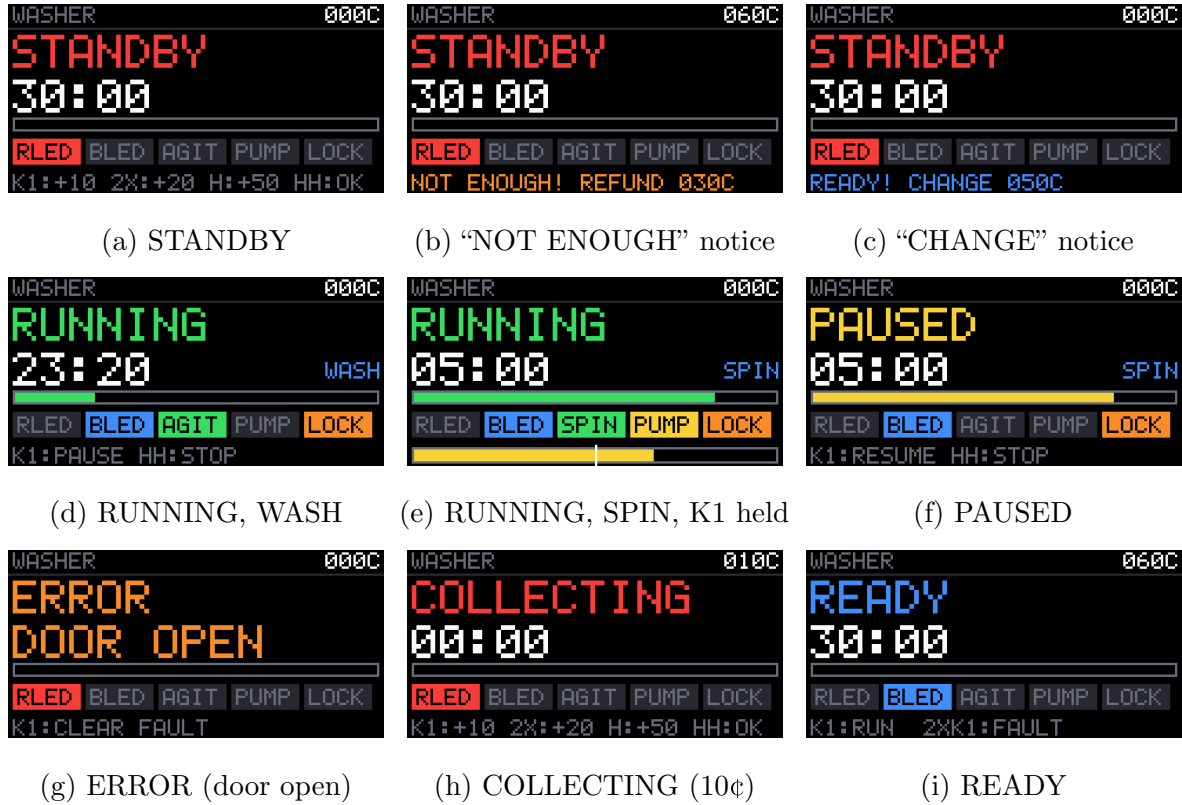


Figure 11.2: LCD dashboard in each state (frames rendered from the firmware UI code)

11.5 Serial terminal log

Each line of the log has the format below (one line per change, at most one per second):

```
1 [12s] STATE=RUNNING PHASE=WASH TIME=00:25 COIN=000C RLED=0 BLED=1 MOTOR=AGIT PUMP=0 LOCK=1 FAULT=-
```

11.6 Resource usage

Table 11.3: Firmware size (arm-none-eabi-size build/h750/washer.elf)

	text	data	bss
Bytes	9 340	4	25 772

The code uses 7.1 % of the 128 KB internal flash. Most of the RAM (bss) is the 25.6 KB LCD frame buffer ($160 \times 80 \times 2$ bytes); the FSM context itself is under 100 bytes.



12 Discussion

12.1 Problems met and how they were solved

Button bouncing. A mechanical contact bounces for several milliseconds, so a raw read would turn one press into several RUN/PAUSE toggles. The integrating debouncer (Section 8) only accepts a new level after 30 ms of stability and emits exactly one latched edge. On the K1 key, the board's RC filter (10 k Ω /100 nF, $\tau = 1$ ms) helps, but the software filter is what makes the behaviour deterministic (HAL-01, TC20).

Timer handling. The first idea of many students is "PAUSE stops the timer". The specification says the opposite. Making the timer a context variable decremented by the *same* handler in RUNNING and PAUSED removed the risk of the two paths diverging. A second subtle point is the end condition: the transition is taken on the tick that goes from 1 to 0, so the cycle lasts exactly T ticks and the FSM never sits in RUNNING with $t = 0$.

Avoiding blocking delays. With HAL_Delay() the LED blinking alone would block the CPU half of the time and inputs would be missed. Every delay was replaced by a counter advanced by the 1 ms tick. The catch-up loop (`while (processed != now)`) keeps the timing exact even when the main loop is temporarily slow, e.g. while a full LCD frame is sent over SPI.

ISR / main-loop race. An earlier version updated the FSM from inside the SysTick ISR while the main loop also dispatched button events, so two contexts could modify the FSM at the same time. The final design keeps the ISR trivial (`g_millis++`) and runs *all* FSM work in the main loop, which removes the race without disabling interrupts.

Double-press STOP. The difficulty is to separate "two presses" from "two unrelated presses". A 1.5 s sliding window implemented with a countdown solves it, and the boundary is tested at 1499 ms and 1501 ms. On the board, a single key also had to produce two STOPs; the two-stage hold (0.6 s, 1.2 s) does this and the second stage is guaranteed to fall inside the window.

One key for seven inputs. The WeAct board exposes only K1 to software (BOOT0 is not a GPIO on the H7). A gesture decoder (click, double-click, two-stage hold) plus



state-dependent mapping gives every command, without changing the FSM. Its cost is a 300 ms delay before a single click is confirmed (to distinguish it from a double-click).

State transitions. Writing the transition table first (Table 4.1) and then one `enter_xxx()` function per state made the LED rules impossible to forget on any incoming transition. Test TC-27 enumerates all (state, event) pairs to check that unspecified events are ignored.

LCD refresh. Redrawing 25.6 KB at every loop iteration would waste time and cause flicker. The firmware builds a small “view” structure and redraws only if it differs from the last drawn one, so the panel is refreshed at most a few times per second. The backlight faded out when driven as a plain GPIO on this board; driving PE10 with a 10 kHz PWM from TIM1 fixed it.

12.2 Limitations

- **No real coin acceptor.** Coins are simulated with K1 gestures, external buttons or UART keys. The pulse-train decoder for a real acceptor is implemented and unit-tested but not wired.
- **No real motor or relays.** Motor, pump, valve and door-lock commands are only displayed. A real machine would need relay/driver hardware and feedback sensors.
- **Only one physical LED.** RLED and BLED are drawn on the LCD; the PE3 LED shows their OR. Two separate LEDs could be added on free GPIOs without software changes (only the callbacks).
- **Error state is simulated.** Faults are injected by a K1 double-click; there is no real door switch, water-level or current sensor.
- **Time scaling for the demo.** The default board build uses a 30 s cycle so the whole cycle can be shown; the 30-minute behaviour is verified by the host tests and by the `FULL_CYCLE=1` build.
- **No power-loss persistence.** A reset during a paid cycle loses the remaining time; a real product would store it in backup SRAM or flash.
- **Front-panel extensions.** The coin staging/confirm step and the “change” notice on the board go beyond the statement (which says READY as soon as 50¢ is reached



and no refund). They are a UI layer only; the FSM follows the statement and its tests are unchanged.

13 Conclusion

The team designed and implemented the control unit of a coin-operated washing machine on the WeAct STM32H750 board. In summary:

- **FSM.** A 6-state extended FSM (STANDBY, COLLECTING, READY, RUNNING, PAUSED, ERROR) with 22 documented transitions was designed from the requirements and implemented as portable C99 code with no hardware dependency.
- **Coin accumulation.** 10/20/50¢ coins are accumulated; READY is reached at 50¢; the balance is cleared on RUN without refund.
- **RUN / PAUSE / STOP.** RUN starts and resumes, PAUSE suspends the actuators, and STOP requires two presses within 1.5 s.
- **Timer.** The 30-minute countdown is derived from a 1 ms SysTick without any blocking delay and keeps counting while paused; the cycle ends automatically.
- **LED indication.** RLED/BLED follow the specification (ON, 1 Hz, 2 Hz) through a non-blocking blinker.
- **Verification.** 94 automated test cases pass on the PC with the same source code as the firmware, and the main scenarios were demonstrated on the real board (Figure 11.1 and the demo video).

Possible extensions are a real pulse-output coin acceptor on a GPIO (the decoder already exists), relay outputs for the motor/pump/valve/lock, real door and water-level sensors for the ERROR state, and saving the remaining time to backup memory to survive a power loss.



References

- [1] *STM32H750VB, STM32H750ZB, STM32H750IB, STM32H750XB: 32-bit Arm Cortex-M7 480 MHz MCUs, 128 Kbyte Flash, 1 Mbyte RAM – Datasheet (DS12556)*. STMicroelectronics, . URL <https://www.st.com/resource/en/datasheet/stm32h750vb.pdf>.
- [2] *RM0433 Reference Manual: STM32H742, STM32H743/753 and STM32H750 Value line advanced Arm-based 32-bit MCUs*. STMicroelectronics, . URL https://www.st.com/resource/en/reference_manual/rm0433-stm32h742-stm32h743753-and-stm32h750-value-line-advanced-armbased-32bit-mcus-stmicroelectronics.pdf.
- [3] WeAct Studio. MiniSTM32H7xx: WeAct studio STM32H7xx core board – schematic, examples and SDK. GitHub repository. URL <https://github.com/WeActStudio/MiniSTM32H7xx>.
- [4] *ST7735S: 132RGB x 162dot 262K Color with Frame Memory Single-Chip TFT Controller/Driver – Datasheet*. Sitronix Technology Corp.
- [5] *UM2217: Description of STM32H7 HAL and low-layer drivers*. STMicroelectronics, . URL https://www.st.com/resource/en/user_manual/um2217-description-of-stm32h7-hal-and-lowlayer-drivers-stmicroelectronics.pdf.
- [6] *Armv7-M Architecture Reference Manual (DDI0403)*. Arm Limited. URL <https://developer.arm.com/documentation/ddi0403/latest>.
- [7] Võ Hoàng Nguyên, Nguyễn Quốc Thắng, and Ngô Nguyễn Thành Nhân. CO3053-BTL2-Washing-Machine: source code of this assignment. Git repository. URL <https://github.com/nguyenchuyenhoa14-hub/CO3053-BTL2-Washing-Machine>.

A Source code

The complete project is available in the team repository^[7]: <https://github.com/nguyenchuyenhoa14-hub/C03053-BTL2-Washing-Machine>. Directory layout:

```
1 src/include/   washing_machine_config.h, washing_machine_fsm.h, hal_interfaces.h
2 src/fsm/       washing_machine_fsm.c           portable FSM (core logic)
3 src/hal/       hal_button_engine.c             debounce + coin pulse decoder
4               hal_led_blinker.c               1 Hz / 2 Hz blinker + actuator interlock
5               hal_button_gesture.c            click / double / hold decoder
6 src/app/       wm_single_button.c             gesture -> FSM event mapping
7               wm_two_button.c                coin staging front panel
8 board/weact_h750/
9               startup_stm32h750.c, stm32h750vb_flash.ld  startup + linker script
10              board_h750.c                     clocks 400 MHz, GPIO, SysTick, USART1
11              lcd_st7735.c, gfx.c, ui_render.c, ui_text.c LCD + UART UI
12              main_h750.c                       firmware entry point (super-loop)
13 tests/        test_washing_machine.c, test_hal_engines.c, test_gesture.c,
14              test_two_button.c, test_ui_text.c
15 sim/          sim_interactive.c (CLI simulator), wokwi/ (web simulation)
```

Build and flash: `make test test_hal test_gesture` (PC tests), `make h750` (firmware), `make flash` (USB-DFU or ST-Link), `make preview` (LCD frames on the PC).

A.1 FSM

Listing 11: `washing_machine_config.h`

```
1 /**
2  * @file washing_machine_config.h
3  * @brief System configuration constants for Washing Machine Control Unit (C03053 -
4  *       BTL 2)
5  */
6 #ifndef WASHING_MACHINE_CONFIG_H
7 #define WASHING_MACHINE_CONFIG_H
8
9 #include <stdint.h>
10
11 #ifdef __cplusplus
12 extern "C" {
13 #endif
14
15 /**
16  * @brief Full cycle duration in seconds (30 minutes = 1800s as per specification).
17  */
```



```
18 #define WM_CYCLE_DURATION_SEC          (1800U)
19
20 /**
21  * @brief Scaled cycle duration used during accelerated unit testing (30s).
22  */
23 #define WM_CYCLE_DURATION_TEST_SEC      (30U)
24
25 /**
26  * @brief Minimum accumulated coin balance required to reach READY state (50 cents).
27  */
28 #define WM_COIN_THRESHOLD_CENTS        (50U)
29
30 /**
31  * @brief Time window to register a double-press on the STOP button (1500 ms = 1.5s).
32  */
33 #define WM_DOUBLE_PRESS_WINDOW_MS      (1500U)
34
35 /**
36  * @brief Software debounce filter duration for digital button inputs (30 ms).
37  */
38 #define WM_DEBOUNCE_TIME_MS            (30U)
39
40 /**
41  * @brief LED blinking period in RUNNING state (1.0 Hz -> 500ms ON / 500ms OFF).
42  */
43 #define WM_BLED_BLINK_PERIOD_MS        (1000U)
44
45 /**
46  * @brief LED blinking period in ERROR state (2.0 Hz -> 250ms ON / 250ms OFF).
47  */
48 #define WM_RLED_BLINK_PERIOD_MS        (500U)
49
50 /**
51  * @brief Fault recovery policy (1 = resume, 0 = legacy "abandon").
52  * @details 1: the state before a fault is remembered. Deposits are kept
53  *           (STANDBY/COLLECTING/READY return
54  *           to themselves); a RUNNING/PAUSED cycle returns to PAUSED when the fault
55  *           is cleared and its
56  *           timer keeps counting during the fault (same rule as PAUSE). 0: clearing a
57  *           fault always
58  *           returns to STANDBY with the deposit and the cycle discarded.
59  */
60 #ifndef WM_ERROR_RESUMES_CYCLE
```



```
58 #define WM_ERROR_RESUMES_CYCLE          (1)
59 #endif
60
61 /**
62  * @brief Diagnostic bitmasks for safety faults (Sensor inputs)
63  */
64 #define WM_FAULT_NONE                    (0x00U)
65 #define WM_FAULT_DOOR_OPEN              (0x01U) /**< Safety interlock open during
        spin/wash */
66 #define WM_FAULT_WATER_TIMEOUT          (0x02U) /**< Inlet water filling timeout */
67 #define WM_FAULT_MOTOR_OVERCURRENT     (0x04U) /**< Motor stall / overcurrent
        detected */
68
69 #ifdef __cplusplus
70 }
71 #endif
72
73 #endif /* WASHING_MACHINE_CONFIG_H */
```

Listing 12: washing_machine_fsm.h

```
1 /**
2  * @file washing_machine_fsm.h
3  * @brief Public interface of the Washing Machine Finite State Machine (FSM)
4  *       CO3053 - Embedded Systems - Assignment 2 (BTL 2)
5  */
6
7 #ifndef WASHING_MACHINE_FSM_H
8 #define WASHING_MACHINE_FSM_H
9
10 #include <stdint.h>
11 #include <stdbool.h>
12 #include "washing_machine_config.h"
13 #include "hal_interfaces.h"
14
15 #ifdef __cplusplus
16 extern "C" {
17 #endif
18
19 /**
20  * @brief States of the Washing Machine Control Unit
21  */
22 typedef enum {
```

```
23 WM_STATE_STANDBY = 0,    /**< Available to serve, no money inserted; RLED=ON,
    BLED=OFF */
24 WM_STATE_COLLECTING,    /**< 0 < deposit < 50¢; RLED=ON, BLED=OFF (STOP x2
    cancels and returns the coins) */
25 WM_STATE_READY,        /**< Deposit >= 50¢; RLED=OFF, BLED=ON */
26 WM_STATE_RUNNING,      /**< Actuators working; BLED=BLINK(1Hz), Timer ticking */
27 WM_STATE_PAUSED,       /**< Actuators stopped; Timer continues ticking down */
28 WM_STATE_ERROR         /**< Hardware/safety error; RLED=BLINK(2Hz), Actuators
    stopped */
29 } wm_state_t;
30
31 /**
32  * @brief Sub-phases of the active washing cycle
33  */
34 typedef enum {
35     WM_PHASE_IDLE = 0,    /**< Machine not washing */
36     WM_PHASE_WASH_AGITATE, /**< Main wash agitation (drum reversing) */
37     WM_PHASE_FINAL_SPIN   /**< High-speed spin dry and wastewater drainage */
38 } wm_cycle_phase_t;
39
40 /**
41  * @brief External events injected into the FSM
42  */
43 typedef enum {
44     WM_EVT_NONE = 0,
45     WM_EVT_COIN_10,    /**< 10-cent coin deposited */
46     WM_EVT_COIN_20,    /**< 20-cent coin deposited */
47     WM_EVT_COIN_50,    /**< 50-cent coin deposited */
48     WM_EVT_BTN_RUN,     /**< RUN button clicked */
49     WM_EVT_BTN_PAUSE,   /**< PAUSE button clicked */
50     WM_EVT_BTN_STOP,    /**< STOP button clicked (single pulse) */
51     WM_EVT_TIMER_TICK_1S, /**< 1-second system clock tick */
52     WM_EVT_TIMER_TICK_1MS, /**< 1-millisecond system tick for
    debounce/double-press */
53     WM_EVT_FAULT_OCCURRED, /**< Safety fault triggered (e.g. lid open during
    spin) */
54     WM_EVT_FAULT_CLEARED  /**< Fault cleared / reset button pressed */
55 } wm_event_t;
56
57 /**
58  * @brief Complete state context of the Washing Machine Control Unit
59  */
60 typedef struct {
```

```
61     wm_state_t state;                /**< Current FSM state */
62     uint32_t coin_balance_cents;      /**< Accumulated coin deposit */
63     uint32_t remaining_cycle_sec;     /**< Countdown timer in seconds (starts at
        1800s) */
64     uint32_t stop_press_count;        /**< Tracks consecutive presses of the STOP
        button */
65     uint32_t stop_window_timer_ms;    /**< Countdown timer for the 1.5s
        double-click window */
66     uint32_t active_error_flags;      /**< Bitmask of active system faults */
67     hal_output_callbacks_t callbacks; /**< HAL output driver callbacks */
68     uint32_t cycle_duration_setting;  /**< Configurable total cycle time (default
        1800s) */
69     wm_state_t state_before_error;    /**< State to return to when a fault is
        cleared (see WM_ERROR_RESUMES_CYCLE) */
70 } wm_context_t;
71
72 /**
73  * @brief Initialize the washing machine FSM context to default STANDBY state.
74  * @param ctx Pointer to FSM context structure.
75  * @param callbacks Pointer to HAL output callback table (can be NULL for headless).
76  */
77 void wm_fsm_init(wm_context_t *ctx, const hal_output_callbacks_t *callbacks);
78
79 /**
80  * @brief Dispatch an event to the FSM.
81  * @param ctx Pointer to FSM context structure.
82  * @param event The event to process.
83  * @return true if state changed or an action was executed; false if ignored.
84  */
85 bool wm_fsm_dispatch_event(wm_context_t *ctx, wm_event_t event);
86
87 /**
88  * @brief Periodic 1 ms tick handler for double-press window and debouncing.
89  * @param ctx Pointer to FSM context structure.
90  */
91 void wm_fsm_tick_1ms(wm_context_t *ctx);
92
93 /**
94  * @brief Periodic 1 second tick handler for cycle countdown.
95  * @param ctx Pointer to FSM context structure.
96  */
97 void wm_fsm_tick_1s(wm_context_t *ctx);
98
```



```
99 /**
100  * @brief Get current FSM state (safe getter).
101  */
102 wm_state_t wm_fsm_get_state(const wm_context_t *ctx);
103
104 /**
105  * @brief Get accumulated coin balance in cents.
106  */
107 uint32_t wm_fsm_get_balance(const wm_context_t *ctx);
108
109 /**
110  * @brief Get remaining cycle time in seconds.
111  */
112 uint32_t wm_fsm_get_remaining_seconds(const wm_context_t *ctx);
113
114 /**
115  * @brief Get active fault bitmask flags.
116  */
117 uint32_t wm_fsm_get_fault_flags(const wm_context_t *ctx);
118
119 /**
120  * @brief Trigger a specific hardware/safety fault bitmask.
121  * @param ctx Pointer to FSM context structure.
122  * @param fault_mask Bitmask of faults (e.g. WM_FAULT_DOOR_OPEN).
123  */
124 void wm_fsm_trigger_fault(wm_context_t *ctx, uint32_t fault_mask);
125
126 /**
127  * @brief Clear all active safety faults and recover towards STANDBY.
128  * @param ctx Pointer to FSM context structure.
129  */
130 void wm_fsm_clear_fault(wm_context_t *ctx);
131
132 /**
133  * @brief Check if the FSM can legally accept a given event in its current state.
134  * @param ctx Pointer to FSM context structure.
135  * @param event The candidate event to query.
136  * @return true if the event would be handled or trigger a transition; false if
137  *         ignored.
138  */
139 bool wm_fsm_can_accept_event(const wm_context_t *ctx, wm_event_t event);
140 /**
```

```
141  * @brief Get current washing cycle sub-phase (Agitate vs Spin dry).
142  * @param ctx Pointer to FSM context structure.
143  * @return Current cycle phase enum.
144  */
145  wm_cycle_phase_t wm_fsm_get_cycle_phase(const wm_context_t *ctx);
146
147  /**
148  * @brief Convert cycle phase enum to human-readable string.
149  */
150  const char* wm_cycle_phase_to_str(wm_cycle_phase_t phase);
151
152  /**
153  * @brief Convert state enum to human-readable string.
154  */
155  const char* wm_state_to_str(wm_state_t state);
156
157  /**
158  * @brief Convert event enum to human-readable string.
159  */
160  const char* wm_event_to_str(wm_event_t event);
161
162  /**
163  * @brief Convert fault bitmask to human-readable diagnostic string.
164  */
165  const char* wm_fault_to_str(uint32_t fault_mask);
166
167  #ifdef __cplusplus
168  }
169  #endif
170
171  #endif /* WASHING_MACHINE_FSM_H */
```

Listing 13: washing_machine_fsm.c

```
1  /**
2  * @file washing_machine_fsm.c
3  * @brief Implementation of the Washing Machine Finite State Machine (FSM)
4  *        CO3053 - Embedded Systems - Assignment 2 (BTL 2)
5  */
6
7  #include "washing_machine_fsm.h"
8  #include <stddef.h>
9  #include <limits.h>
```



```
10
11 /* Helper macro for safe callback execution */
12 #define CALL_HAL(cb, func, ...) do { if ((cb)->func) { (cb)->func(__VA_ARGS__); } }
    while(0)
13 #define CALL_HAL_VOID(cb, func) do { if ((cb)->func) { (cb)->func(); } } while(0)
14
15 static void enter_standby(wm_context_t *ctx) {
16     ctx->state = WM_STATE_STANDBY;
17     ctx->coin_balance_cents = 0;
18     ctx->remaining_cycle_sec = 0;
19     ctx->stop_press_count = 0;
20     ctx->stop_window_timer_ms = 0;
21
22     CALL_HAL(&ctx->callbacks, set_motor, HAL_MOTOR_OFF);
23     CALL_HAL(&ctx->callbacks, set_water_valve, false);
24     CALL_HAL(&ctx->callbacks, set_drain_pump, false);
25     CALL_HAL(&ctx->callbacks, set_door_lock, false);
26     CALL_HAL(&ctx->callbacks, set_bled, HAL_LED_OFF);
27     CALL_HAL(&ctx->callbacks, set_rled, HAL_LED_ON);
28 }
29
30 static void enter_ready(wm_context_t *ctx) {
31     ctx->state = WM_STATE_READY;
32     CALL_HAL(&ctx->callbacks, set_rled, HAL_LED_OFF);
33     CALL_HAL(&ctx->callbacks, set_bled, HAL_LED_ON);
34 }
35
36 static void enter_collecting(wm_context_t *ctx) {
37     ctx->state = WM_STATE_COLLECTING;
38     ctx->stop_press_count = 0;
39     ctx->stop_window_timer_ms = 0;
40     CALL_HAL(&ctx->callbacks, set_rled, HAL_LED_ON);
41     CALL_HAL(&ctx->callbacks, set_bled, HAL_LED_OFF);
42 }
43
44 #if WM_ERROR_RESUMES_CYCLE
45 /* Re-enter the idle-side state that matches the deposit WITHOUT touching the deposit
    */
46 static void restore_idle(wm_context_t *ctx) {
47     if (ctx->coin_balance_cents >= WM_COIN_THRESHOLD_CENTS) {
48         enter_ready(ctx);
49     } else if (ctx->coin_balance_cents > 0) {
50         enter_collecting(ctx);
```

```
51     } else {
52         enter_standby(ctx);
53     }
54 }
55
56 static bool cycle_in_progress(const wm_context_t *ctx) {
57     return ctx->state_before_error == WM_STATE_RUNNING || ctx->state_before_error ==
58     WM_STATE_PAUSED;
59 }
60 #endif
61
62 static void update_running_actuators(wm_context_t *ctx) {
63     uint32_t total = (ctx->cycle_duration_setting > 0) ?
64         ctx->cycle_duration_setting : WM_CYCLE_DURATION_SEC;
65     uint32_t spin_threshold = total / 6; /* Final 1/6th of wash cycle is spin dry */
66
67     CALL_HAL(&ctx->callbacks, set_door_lock, true);
68     if (ctx->remaining_cycle_sec <= spin_threshold) {
69         /* Final drain & high-speed spin dry */
70         CALL_HAL(&ctx->callbacks, set_drain_pump, true);
71         CALL_HAL(&ctx->callbacks, set_water_valve, false);
72         CALL_HAL(&ctx->callbacks, set_motor, HAL_MOTOR_SPIN);
73     } else {
74         /* Main wash agitation */
75         CALL_HAL(&ctx->callbacks, set_drain_pump, false);
76         CALL_HAL(&ctx->callbacks, set_water_valve, false);
77         CALL_HAL(&ctx->callbacks, set_motor, HAL_MOTOR_AGITATE);
78     }
79 }
80
81 static void enter_running(wm_context_t *ctx, bool is_resuming) {
82     ctx->state = WM_STATE_RUNNING;
83     ctx->stop_press_count = 0;
84     ctx->stop_window_timer_ms = 0;
85
86     if (!is_resuming) {
87         /* Unconditional clearance of money without returning redundancies */
88         ctx->coin_balance_cents = 0;
89         /* Activate 30-minute countdown timer */
90         ctx->remaining_cycle_sec = (ctx->cycle_duration_setting > 0) ?
91             ctx->cycle_duration_setting :
92             WM_CYCLE_DURATION_SEC;
93     }
94 }
```



```
92
93     CALL_HAL(&ctx->callbacks, set_rled, HAL_LED_OFF);
94     CALL_HAL(&ctx->callbacks, set_bled, HAL_LED_BLINK_1HZ);
95     update_running_actuators(ctx);
96 }
97
98 static void enter_paused(wm_context_t *ctx) {
99     ctx->state = WM_STATE_PAUSED;
100     ctx->stop_press_count = 0;
101     ctx->stop_window_timer_ms = 0;
102
103     /* Actuators suspended, but remaining_cycle_sec continues counting down! */
104     CALL_HAL(&ctx->callbacks, set_motor, HAL_MOTOR_OFF);
105     CALL_HAL(&ctx->callbacks, set_water_valve, false);
106     CALL_HAL(&ctx->callbacks, set_drain_pump, false);
107     CALL_HAL(&ctx->callbacks, set_rled, HAL_LED_OFF);
108     CALL_HAL(&ctx->callbacks, set_bled, HAL_LED_ON);
109 }
110
111 static void enter_error(wm_context_t *ctx) {
112     if (ctx->state != WM_STATE_ERROR) {
113         ctx->state_before_error = ctx->state; /* a second fault keeps the original
114         context */
115     }
116     ctx->state = WM_STATE_ERROR;
117     ctx->stop_press_count = 0;
118     ctx->stop_window_timer_ms = 0;
119
120     CALL_HAL(&ctx->callbacks, set_motor, HAL_MOTOR_OFF);
121     CALL_HAL(&ctx->callbacks, set_water_valve, false);
122     CALL_HAL(&ctx->callbacks, set_drain_pump, false);
123     CALL_HAL(&ctx->callbacks, set_door_lock, false);
124     CALL_HAL(&ctx->callbacks, set_bled, HAL_LED_OFF);
125     CALL_HAL(&ctx->callbacks, set_rled, HAL_LED_BLINK_2HZ);
126 }
127
128 void wm_fsm_init(wm_context_t *ctx, const hal_output_callbacks_t *callbacks) {
129     if (!ctx) {
130         return;
131     }
132
133     ctx->coin_balance_cents = 0;
134     ctx->remaining_cycle_sec = 0;
```



```
134     ctx->stop_press_count = 0;
135     ctx->stop_window_timer_ms = 0;
136     ctx->active_error_flags = WM_FAULT_NONE;
137     ctx->cycle_duration_setting = WM_CYCLE_DURATION_SEC;
138     ctx->state_before_error = WM_STATE_STANDBY;
139
140     if (callbacks) {
141         ctx->callbacks = *callbacks;
142     } else {
143         /* Initialize empty callbacks */
144         ctx->callbacks.set_rled = NULL;
145         ctx->callbacks.set_bled = NULL;
146         ctx->callbacks.set_motor = NULL;
147         ctx->callbacks.set_water_valve = NULL;
148         ctx->callbacks.set_drain_pump = NULL;
149         ctx->callbacks.set_door_lock = NULL;
150         ctx->callbacks.on_cycle_complete = NULL;
151         ctx->callbacks.return_coins = NULL;
152     }
153
154     enter_standby(ctx);
155 }
156
157 static bool handle_coin_deposit(wm_context_t *ctx, uint32_t amount) {
158     if (ctx->state == WM_STATE_STANDBY || ctx->state == WM_STATE_COLLECTING) {
159         if (ctx->coin_balance_cents <= (UINT32_MAX - amount)) {
160             ctx->coin_balance_cents += amount;
161         }
162         if (ctx->coin_balance_cents >= WM_COIN_THRESHOLD_CENTS) {
163             enter_ready(ctx);
164         } else {
165             enter_collecting(ctx);
166         }
167         return true;
168     } else if (ctx->state == WM_STATE_READY) {
169         /* Surplus money is accepted, but will be cleared upon RUN without refund */
170         if (ctx->coin_balance_cents <= (UINT32_MAX - amount)) {
171             ctx->coin_balance_cents += amount;
172         }
173         return true;
174     }
175     /* Ignored in RUNNING, PAUSED, ERROR */
176     return false;
```

```
177 }
178
179 static bool handle_stop_button(wm_context_t *ctx) {
180     if (ctx->state == WM_STATE_STANDBY) {
181         return false;
182     }
183
184     if (ctx->stop_press_count == 0) {
185         /* First press: start double-click window */
186         ctx->stop_press_count = 1;
187         ctx->stop_window_timer_ms = WM_DOUBLE_PRESS_WINDOW_MS;
188         return true;
189     } else {
190         /* Second press within window: Force Stop confirmed! */
191         ctx->stop_press_count = 0;
192         ctx->stop_window_timer_ms = 0;
193         /* User cancel before RUN: give the deposit back instead of silently
194         discarding it.
195         * (RUN itself still follows the zero-refund policy: surplus is never
196         returned.) */
197         if ((ctx->state == WM_STATE_READY || ctx->state == WM_STATE_COLLECTING) &&
198             ctx->coin_balance_cents > 0) {
199             CALL_HAL(&ctx->callbacks, return_coins, ctx->coin_balance_cents);
200         }
201         ctx->coin_balance_cents = 0;
202         enter_standby(ctx);
203         return true;
204     }
205 }
206
207 static bool handle_1s_timer_tick(wm_context_t *ctx) {
208     if (ctx->remaining_cycle_sec > 1) {
209         ctx->remaining_cycle_sec--;
210         if (ctx->state == WM_STATE_RUNNING) {
211             update_running_actuators(ctx);
212         }
213         return true;
214     } else if (ctx->remaining_cycle_sec == 1) {
215         ctx->remaining_cycle_sec = 0;
216         CALL_HAL_VOID(&ctx->callbacks, on_cycle_complete);
217         enter_standby(ctx);
218         return true;
219     }
220 }
```



```
218     return false;
219 }
220
221 #if WM_ERROR_RESUMES_CYCLE
222 /* Timer keeps counting during a fault when a cycle was in progress (same rule as
223 PAUSE) */
224 static bool handle_error_tick(wm_context_t *ctx) {
225     if (!cycle_in_progress(ctx)) {
226         return false;
227     }
228     if (ctx->remaining_cycle_sec > 1) {
229         ctx->remaining_cycle_sec--;
230         return true;
231     }
232     if (ctx->remaining_cycle_sec == 1) {
233         ctx->remaining_cycle_sec = 0;
234         ctx->state_before_error = WM_STATE_STANDBY; /* cycle finished while faulted
235         */
236         CALL_HAL_VOID(&ctx->callbacks, on_cycle_complete);
237         return true;
238     }
239     return false;
240 }
241 #endif
242
243 static bool handle_fault_cleared(wm_context_t *ctx) {
244     ctx->active_error_flags = WM_FAULT_NONE;
245     #if WM_ERROR_RESUMES_CYCLE
246     if (cycle_in_progress(ctx) && ctx->remaining_cycle_sec > 0) {
247         enter_paused(ctx); /* safe: the user presses RUN to continue */
248     } else {
249         if (cycle_in_progress(ctx)) {
250             ctx->coin_balance_cents = 0;
251         }
252         restore_idle(ctx); /* deposit is kept */
253     }
254 #else
255     ctx->coin_balance_cents = 0;
256     enter_standby(ctx);
257 #endif
258     return true;
259 }
```



```
259 bool wm_fsm_dispatch_event(wm_context_t *ctx, wm_event_t event) {
260     if (!ctx) {
261         return false;
262     }
263
264     /* Global emergency error transition */
265     if (event == WM_EVT_FAULT_OCCURRED) {
266         if (ctx->active_error_flags == WM_FAULT_NONE) {
267             ctx->active_error_flags = WM_FAULT_DOOR_OPEN;
268         }
269         if (ctx->state != WM_STATE_ERROR) {
270             enter_error(ctx);
271             return true;
272         }
273         return false;
274     }
275
276     /* System 1ms tick handling */
277     if (event == WM_EVT_TIMER_TICK_1MS) {
278         wm_fsm_tick_1ms(ctx);
279         return true;
280     }
281
282     /* Dispatch based on current state */
283     switch (ctx->state) {
284         case WM_STATE_STANDBY:
285             switch (event) {
286                 case WM_EVT_COIN_10:
287                     return handle_coin_deposit(ctx, 10);
288                 case WM_EVT_COIN_20:
289                     return handle_coin_deposit(ctx, 20);
290                 case WM_EVT_COIN_50:
291                     return handle_coin_deposit(ctx, 50);
292                 default:
293                     /* All button presses ignored in Standby */
294                     return false;
295             }
296
297         case WM_STATE_COLLECTING:
298             switch (event) {
299                 case WM_EVT_COIN_10:
300                     return handle_coin_deposit(ctx, 10);
301                 case WM_EVT_COIN_20:
```



```
302         return handle_coin_deposit(ctx, 20);
303     case WM_EVT_COIN_50:
304         return handle_coin_deposit(ctx, 50);
305     case WM_EVT_BTN_STOP:
306         return handle_stop_button(ctx);
307     default:
308         return false;
309 }
310
311 case WM_STATE_READY:
312     switch (event) {
313     case WM_EVT_COIN_10:
314         return handle_coin_deposit(ctx, 10);
315     case WM_EVT_COIN_20:
316         return handle_coin_deposit(ctx, 20);
317     case WM_EVT_COIN_50:
318         return handle_coin_deposit(ctx, 50);
319     case WM_EVT_BTN_RUN:
320         enter_running(ctx, false);
321         return true;
322     case WM_EVT_BTN_STOP:
323         return handle_stop_button(ctx);
324     default:
325         return false;
326     }
327
328 case WM_STATE_RUNNING:
329     switch (event) {
330     case WM_EVT_BTN_PAUSE:
331         enter_paused(ctx);
332         return true;
333     case WM_EVT_BTN_STOP:
334         return handle_stop_button(ctx);
335     case WM_EVT_TIMER_TICK_1S:
336         return handle_1s_timer_tick(ctx);
337     default:
338         return false;
339     }
340
341 case WM_STATE_PAUSED:
342     switch (event) {
343     case WM_EVT_BTN_RUN:
344         enter_running(ctx, true);
```

```
345         return true;
346     case WM_EVT_BTN_STOP:
347         return handle_stop_button(ctx);
348     case WM_EVT_TIMER_TICK_1S:
349         /* CRITICAL REQUIREMENT: Timer continues counting down in PAUSED
350         state! */
351         return handle_1s_timer_tick(ctx);
352     default:
353         return false;
354 }
355
356 case WM_STATE_ERROR:
357     if (event == WM_EVT_FAULT_CLEARED) {
358         return handle_fault_cleared(ctx);
359     }
360 #if WM_ERROR_RESUMES_CYCLE
361     if (event == WM_EVT_TIMER_TICK_1S) {
362         return handle_error_tick(ctx);
363     }
364 #endif
365     return false;
366
367     default:
368         return false;
369 }
370
371 void wm_fsm_tick_1ms(wm_context_t *ctx) {
372     if (!ctx) {
373         return;
374     }
375
376     /* Decrement double-press sliding window */
377     if (ctx->stop_window_timer_ms > 0) {
378         ctx->stop_window_timer_ms--;
379         if (ctx->stop_window_timer_ms == 0) {
380             /* Double-press window expired: discard single press */
381             ctx->stop_press_count = 0;
382         }
383     }
384 }
385
386 void wm_fsm_tick_1s(wm_context_t *ctx) {
```



```
387     wm_fsm_dispatch_event(ctx, WM_EVT_TIMER_TICK_1S);
388 }
389
390 const char* wm_state_to_str(wm_state_t state) {
391     switch (state) {
392         case WM_STATE_STANDBY: return "STANDBY";
393         case WM_STATE_COLLECTING: return "COLLECTING";
394         case WM_STATE_READY: return "READY";
395         case WM_STATE_RUNNING: return "RUNNING";
396         case WM_STATE_PAUSED: return "PAUSED";
397         case WM_STATE_ERROR: return "ERROR";
398         default: return "UNKNOWN";
399     }
400 }
401
402 const char* wm_event_to_str(wm_event_t event) {
403     switch (event) {
404         case WM_EVT_NONE: return "EVT_NONE";
405         case WM_EVT_COIN_10: return "COIN_10c";
406         case WM_EVT_COIN_20: return "COIN_20c";
407         case WM_EVT_COIN_50: return "COIN_50c";
408         case WM_EVT_BTN_RUN: return "BTN_RUN";
409         case WM_EVT_BTN_PAUSE: return "BTN_PAUSE";
410         case WM_EVT_BTN_STOP: return "BTN_STOP";
411         case WM_EVT_TIMER_TICK_1S: return "TICK_1S";
412         case WM_EVT_TIMER_TICK_1MS: return "TICK_1MS";
413         case WM_EVT_FAULT_OCCURRED: return "FAULT_OCCURRED";
414         case WM_EVT_FAULT_CLEARED: return "FAULT_CLEARED";
415         default: return "UNKNOWN_EVENT";
416     }
417 }
418
419 wm_state_t wm_fsm_get_state(const wm_context_t *ctx) {
420     return ctx ? ctx->state : WM_STATE_STANDBY;
421 }
422
423 uint32_t wm_fsm_get_balance(const wm_context_t *ctx) {
424     return ctx ? ctx->coin_balance_cents : 0;
425 }
426
427 uint32_t wm_fsm_get_remaining_seconds(const wm_context_t *ctx) {
428     return ctx ? ctx->remaining_cycle_sec : 0;
429 }
```



```
430
431 uint32_t wm_fsm_get_fault_flags(const wm_context_t *ctx) {
432     return ctx ? ctx->active_error_flags : 0;
433 }
434
435 void wm_fsm_trigger_fault(wm_context_t *ctx, uint32_t fault_mask) {
436     if (!ctx) {
437         return;
438     }
439     ctx->active_error_flags |= fault_mask;
440     enter_error(ctx);
441 }
442
443 void wm_fsm_clear_fault(wm_context_t *ctx) {
444     if (!ctx) {
445         return;
446     }
447     ctx->active_error_flags = WM_FAULT_NONE;
448     wm_fsm_dispatch_event(ctx, WM_EVT_FAULT_CLEARED);
449 }
450
451 const char* wm_fault_to_str(uint32_t fault_mask) {
452     if (fault_mask == WM_FAULT_NONE) return "NO_FAULT";
453     if (fault_mask & WM_FAULT_DOOR_OPEN) return "DOOR_LATCH_OPEN";
454     if (fault_mask & WM_FAULT_WATER_TIMEOUT) return "WATER_INLET_TIMEOUT";
455     if (fault_mask & WM_FAULT_MOTOR_OVERCURRENT) return "MOTOR_OVERCURRENT";
456     return "MULTIPLE_FAULTS";
457 }
458
459 bool wm_fsm_can_accept_event(const wm_context_t *ctx, wm_event_t event) {
460     if (!ctx) {
461         return false;
462     }
463
464     if (event == WM_EVT_FAULT_OCCURRED) {
465         return ctx->state != WM_STATE_ERROR;
466     }
467     if (event == WM_EVT_TIMER_TICK_1MS) {
468         return true;
469     }
470
471     switch (ctx->state) {
472         case WM_STATE_STANDBY:
```

```
473     return (event == WM_EVT_COIN_10 || event == WM_EVT_COIN_20 || event ==
WM_EVT_COIN_50);
474
475     case WM_STATE_COLLECTING:
476         return (event == WM_EVT_COIN_10 || event == WM_EVT_COIN_20 || event ==
WM_EVT_COIN_50 ||
477             event == WM_EVT_BTN_STOP);
478
479     case WM_STATE_READY:
480         return (event == WM_EVT_COIN_10 || event == WM_EVT_COIN_20 || event ==
WM_EVT_COIN_50 ||
481             event == WM_EVT_BTN_RUN || event == WM_EVT_BTN_STOP);
482
483     case WM_STATE_RUNNING:
484         return (event == WM_EVT_BTN_PAUSE || event == WM_EVT_BTN_STOP ||
(event == WM_EVT_TIMER_TICK_1S && ctx->remaining_cycle_sec > 0));
485
486     case WM_STATE_PAUSED:
487         return (event == WM_EVT_BTN_RUN || event == WM_EVT_BTN_STOP ||
(event == WM_EVT_TIMER_TICK_1S && ctx->remaining_cycle_sec > 0));
488
489     case WM_STATE_ERROR:
490
491     #if WM_ERROR_RESUMES_CYCLE
492         if (event == WM_EVT_TIMER_TICK_1S) {
493             return cycle_in_progress(ctx) && ctx->remaining_cycle_sec > 0;
494         }
495     #endif
496
497     return (event == WM_EVT_FAULT_CLEARED);
498
499     default:
500         return false;
501 }
502 }
503
504 wm_cycle_phase_t wm_fsm_get_cycle_phase(const wm_context_t *ctx) {
505     if (!ctx) {
506         return WM_PHASE_IDLE;
507     }
508
509     if (ctx->state == WM_STATE_RUNNING || ctx->state == WM_STATE_PAUSED) {
510         uint32_t total = (ctx->cycle_duration_setting > 0) ?
                    ctx->cycle_duration_setting : WM_CYCLE_DURATION_SEC;
511         uint32_t spin_threshold = total / 6;
512     }
```

```
513     if (ctx->remaining_cycle_sec <= spin_threshold && ctx->remaining_cycle_sec >
514         0) {
515         return WM_PHASE_FINAL_SPIN;
516     }
517     return WM_PHASE_WASH_AGITATE;
518 }
519 return WM_PHASE_IDLE;
520 }
521
522 const char* wm_cycle_phase_to_str(wm_cycle_phase_t phase) {
523     switch (phase) {
524         case WM_PHASE_IDLE: return "IDLE";
525         case WM_PHASE_WASH_AGITATE: return "WASH_AGITATE";
526         case WM_PHASE_FINAL_SPIN: return "FINAL_SPIN";
527         default: return "UNKNOWN_PHASE";
528     }
529 }
```

A.2 Input and output engines

Listing 14: hal_button_engine.c

```
1  /**
2   * @file hal_button_engine.c
3   * @brief Implementation of software debouncer and coin pulse validator
4   */
5
6  #include "hal_button_engine.h"
7  #include <stddef.h>
8
9  void hal_button_init(hal_button_t *btn, bool active_low) {
10     if (!btn) {
11         return;
12     }
13     btn->active_low = active_low;
14     btn->debounced_state = false;
15     btn->last_raw_state = active_low ? true : false;
16     btn->debounce_counter = 0;
17     btn->pressed_event = false;
18     btn->released_event = false;
19 }
```



```
20
21 void hal_button_update(hal_button_t *btn, bool raw_pin_high, uint32_t delta_ms) {
22     if (!btn) {
23         return;
24     }
25
26     bool current_pressed = btn->active_low ? (!raw_pin_high) : (raw_pin_high);
27
28     if (current_pressed != btn->debounced_state) {
29         if (delta_ms > (UINT32_MAX - btn->debounce_counter)) {
30             btn->debounce_counter = HAL_DEBOUNCE_DEFAULT_MS;
31         } else {
32             btn->debounce_counter += delta_ms;
33         }
34         if (btn->debounce_counter >= HAL_DEBOUNCE_DEFAULT_MS) {
35             btn->debounced_state = current_pressed;
36             btn->debounce_counter = 0;
37             if (btn->debounced_state) {
38                 btn->pressed_event = true;
39             } else {
40                 btn->released_event = true;
41             }
42         }
43     } else {
44         btn->debounce_counter = 0;
45     }
46
47     btn->last_raw_state = raw_pin_high;
48 }
49
50 bool hal_button_was_pressed(hal_button_t *btn) {
51     if (!btn) {
52         return false;
53     }
54     if (btn->pressed_event) {
55         btn->pressed_event = false;
56         return true;
57     }
58     return false;
59 }
60
61 bool hal_button_was_released(hal_button_t *btn) {
62     if (!btn) {
```



```
63     return false;
64 }
65 if (btn->released_event) {
66     btn->released_event = false;
67     return true;
68 }
69 return false;
70 }
71
72 bool hal_button_is_pressed(const hal_button_t *btn) {
73     return btn ? btn->debounced_state : false;
74 }
75
76 void hal_coin_pulse_init(hal_coin_pulse_detector_t *det, bool active_low, uint32_t
77     timeout_ms) {
78     if (!det) {
79         return;
80     }
81     hal_button_init(&det->pulse_input, active_low);
82     det->pulse_count = 0;
83     det->silence_timer_ms = 0;
84     det->inter_pulse_timeout_ms = (timeout_ms > 0) ? timeout_ms : 150U;
85     det->fifo_head = 0;
86     det->fifo_tail = 0;
87     det->fifo_count = 0;
88     for (uint8_t i = 0; i < HAL_COIN_FIFO_CAPACITY; ++i) {
89         det->fifo[i] = 0;
90     }
91 }
92
93 void hal_coin_pulse_update(hal_coin_pulse_detector_t *det, bool raw_pin_high,
94     uint32_t delta_ms) {
95     if (!det) {
96         return;
97     }
98
99     hal_button_update(&det->pulse_input, raw_pin_high, delta_ms);
100
101     if (hal_button_was_pressed(&det->pulse_input)) {
102         det->pulse_count++;
103         det->silence_timer_ms = 0;
104     } else if (det->pulse_count > 0) {
105         if (delta_ms > (UINT32_MAX - det->silence_timer_ms)) {
```

```
104     det->silence_timer_ms = det->inter_pulse_timeout_ms;
105 } else {
106     det->silence_timer_ms += delta_ms;
107 }
108 if (det->silence_timer_ms >= det->inter_pulse_timeout_ms) {
109     uint32_t coin_value = 0;
110     /* Pulse train finished: decode standard denomination */
111     if (det->pulse_count == 1) {
112         coin_value = 10;
113     } else if (det->pulse_count == 2) {
114         coin_value = 20;
115     } else if (det->pulse_count == 5) {
116         coin_value = 50;
117     }
118
119     /* Push to FIFO if valid and queue has space */
120     if (coin_value > 0 && det->fifo_count < HAL_COIN_FIFO_CAPACITY) {
121         det->fifo[det->fifo_head] = coin_value;
122         det->fifo_head = (uint8_t)((det->fifo_head + 1U) %
HAL_COIN_FIFO_CAPACITY);
123         det->fifo_count++;
124     }
125
126     det->pulse_count = 0;
127     det->silence_timer_ms = 0;
128 }
129 }
130 }
131
132 uint32_t hal_coin_pulse_get_coin(hal_coin_pulse_detector_t *det) {
133     if (!det || det->fifo_count == 0U) {
134         return 0;
135     }
136     uint32_t val = det->fifo[det->fifo_tail];
137     det->fifo_tail = (uint8_t)((det->fifo_tail + 1U) % HAL_COIN_FIFO_CAPACITY);
138     det->fifo_count--;
139     return val;
140 }
141
142 uint8_t hal_coin_pulse_available(const hal_coin_pulse_detector_t *det) {
143     return det ? det->fifo_count : 0U;
144 }
```



Listing 15: hal_led_blinker.c

```
1  /**
2   * @file hal_led_blinker.c
3   * @brief Implementation of non-blocking LED blinker engine and actuator interlock
4   *       guard
5   */
6  #include "hal_led_blinker.h"
7  #include <stddef.h>
8
9  void hal_led_blinker_init(hal_led_blinker_t *blinker, bool active_high) {
10     if (!blinker) {
11         return;
12     }
13     blinker->mode = HAL_LED_OFF;
14     blinker->active_high = active_high;
15     blinker->output_level = active_high ? false : true;
16     blinker->phase_timer_ms = 0;
17     blinker->half_period_ms = 0;
18 }
19
20 void hal_led_blinker_set_mode(hal_led_blinker_t *blinker, hal_led_state_t mode) {
21     if (!blinker) {
22         return;
23     }
24
25     if (blinker->mode == mode) {
26         return;
27     }
28
29     blinker->mode = mode;
30     blinker->phase_timer_ms = 0;
31
32     switch (mode) {
33     case HAL_LED_OFF:
34         blinker->half_period_ms = 0;
35         blinker->output_level = blinker->active_high ? false : true;
36         break;
37
38     case HAL_LED_ON:
39         blinker->half_period_ms = 0;
40         blinker->output_level = blinker->active_high ? true : false;
41         break;
```

```
42
43     case HAL_LED_BLINK_1HZ:
44         blinker->half_period_ms = WM_BLED_BLINK_PERIOD_MS / 2U; /* 500 ms */
45         blinker->output_level = blinker->active_high ? true : false;
46         break;
47
48     case HAL_LED_BLINK_2HZ:
49         blinker->half_period_ms = WM_RLED_BLINK_PERIOD_MS / 2U; /* 250 ms */
50         blinker->output_level = blinker->active_high ? true : false;
51         break;
52
53     default:
54         blinker->half_period_ms = 0;
55         blinker->output_level = blinker->active_high ? false : true;
56         break;
57 }
58 }
59
60 void hal_led_blinker_tick_ms(hal_led_blinker_t *blinker, uint32_t delta_ms) {
61     if (!blinker || blinker->half_period_ms == 0U) {
62         return;
63     }
64
65     if (blinker->mode == HAL_LED_BLINK_1HZ || blinker->mode == HAL_LED_BLINK_2HZ) {
66         /* Guard against wrap-around under extreme delta_ms: reducing modulo one FULL
67         * period (2 half-periods) preserves both the phase and the ON/OFF parity. */
68         if (delta_ms > (UINT32_MAX - blinker->phase_timer_ms)) {
69             delta_ms %= (2U * blinker->half_period_ms);
70         }
71         blinker->phase_timer_ms += delta_ms;
72         if (blinker->phase_timer_ms >= blinker->half_period_ms) {
73             uint32_t toggles = blinker->phase_timer_ms / blinker->half_period_ms;
74             if ((toggles & 1U) != 0U) {
75                 blinker->output_level = !blinker->output_level;
76             }
77             blinker->phase_timer_ms %= blinker->half_period_ms;
78         }
79     }
80 }
81
82 bool hal_led_blinker_get_output(const hal_led_blinker_t *blinker) {
83     return blinker ? blinker->output_level : false;
84 }
```



```
85
86 bool hal_actuator_is_safe(const hal_actuator_guard_t *guard) {
87     if (!guard) {
88         return false;
89     }
90
91     /* Safety Rule 1: Motor must NEVER run if door lock is unengaged */
92     if (guard->motor != HAL_MOTOR_OFF && !guard->door_lock) {
93         return false;
94     }
95
96     /* Safety Rule 2: High-speed spin mode requires drain pump to be active */
97     if (guard->motor == HAL_MOTOR_SPIN && !guard->drain_pump) {
98         return false;
99     }
100
101     /* Safety Rule 3: Water inlet valve must NEVER be open during high-speed spin dry */
102     if (guard->motor == HAL_MOTOR_SPIN && guard->water_valve) {
103         return false;
104     }
105
106     /* Safety Rule 4: Water inlet valve and drain pump must NEVER run simultaneously */
107     if (guard->water_valve && guard->drain_pump) {
108         return false;
109     }
110
111     return true;
112 }
```

Listing 16: hal_button_gesture.c

```
1 /**
2  * @file hal_button_gesture.c
3  * @brief Single-button gesture decoder implementation
4  */
5
6 #include "hal_button_gesture.h"
7
8 void hal_gesture_init(hal_gesture_ctx_t *g) {
9     if (g == NULL) {
10         return;
11     }
12 }
```



```
11     }
12     g->phase = HAL_GST_IDLE;
13     g->timer_ms = 0U;
14     g->hold2_fired = false;
15 }
16
17 hal_gesture_t hal_gesture_update(hal_gesture_ctx_t *g, bool pressed, uint32_t
    delta_ms) {
18     if (g == NULL) {
19         return HAL_GESTURE_NONE;
20     }
21
22     switch (g->phase) {
23         case HAL_GST_IDLE:
24             if (pressed) {
25                 g->phase = HAL_GST_DOWN1;
26                 g->timer_ms = 0U;
27                 g->hold2_fired = false;
28             }
29             break;
30
31         case HAL_GST_DOWN1:
32             g->timer_ms += delta_ms;
33             if (!pressed) {
34                 g->phase = HAL_GST_WAIT;
35                 g->timer_ms = 0U;
36             } else if (g->timer_ms >= HAL_GESTURE_HOLD1_MS) {
37                 g->phase = HAL_GST_HELD;
38                 return HAL_GESTURE_HOLD1;
39             }
40             break;
41
42         case HAL_GST_WAIT:
43             g->timer_ms += delta_ms;
44             if (pressed) {
45                 g->phase = HAL_GST_DOWN2;
46                 g->timer_ms = 0U;
47             } else if (g->timer_ms >= HAL_GESTURE_DOUBLE_GAP_MS) {
48                 g->phase = HAL_GST_IDLE;
49                 return HAL_GESTURE_CLICK;
50             }
51             break;
52     }
```



```
53     case HAL_GST_DOWN2:
54         g->timer_ms += delta_ms;
55         if (!pressed) {
56             g->phase = HAL_GST_IDLE;
57             return HAL_GESTURE_DOUBLE;
58         }
59         if (g->timer_ms >= HAL_GESTURE_HOLD1_MS) {
60             g->phase = HAL_GST_HELD;
61             return HAL_GESTURE_HOLD1;
62         }
63         break;
64
65     case HAL_GST_HELD:
66         g->timer_ms += delta_ms;
67         if (!pressed) {
68             g->phase = HAL_GST_IDLE;
69             if (!g->hold2_fired) {
70                 return HAL_GESTURE_HOLD1_END;
71             }
72         } else if (!g->hold2_fired && g->timer_ms >= HAL_GESTURE_HOLD2_MS) {
73             g->hold2_fired = true;
74             return HAL_GESTURE_HOLD2;
75         }
76         break;
77
78     default:
79         g->phase = HAL_GST_IDLE;
80         break;
81 }
82 return HAL_GESTURE_NONE;
83 }
84
85 uint32_t hal_gesture_hold_ms(const hal_gesture_ctx_t *g) {
86     if (g == NULL) {
87         return 0U;
88     }
89     if (g->phase == HAL_GST_DOWN1 || g->phase == HAL_GST_DOWN2 || g->phase ==
90         HAL_GST_HELD) {
91         return g->timer_ms;
92     }
93     return 0U;
94 }
```



Listing 17: wm_single_button.c

```
1  /**
2   * @file wm_single_button.c
3   * @brief Single-button gesture to FSM event mapping
4   */
5
6  #include "wm_single_button.h"
7
8  static bool apply_click(wm_context_t *ctx) {
9      switch (ctx->state) {
10         case WM_STATE_STANDBY:
11         case WM_STATE_COLLECTING:
12             return wm_fsm_dispatch_event(ctx, WM_EVT_COIN_50);
13         case WM_STATE_READY:
14         case WM_STATE_PAUSED:
15             return wm_fsm_dispatch_event(ctx, WM_EVT_BTN_RUN);
16         case WM_STATE_RUNNING:
17             return wm_fsm_dispatch_event(ctx, WM_EVT_BTN_PAUSE);
18         case WM_STATE_ERROR:
19             wm_fsm_clear_fault(ctx);
20             return true;
21         default:
22             return false;
23     }
24 }
25
26 static bool apply_double(wm_context_t *ctx) {
27     switch (ctx->state) {
28         case WM_STATE_STANDBY:
29         case WM_STATE_COLLECTING:
30             return wm_fsm_dispatch_event(ctx, WM_EVT_COIN_10);
31         case WM_STATE_READY:
32         case WM_STATE_RUNNING:
33         case WM_STATE_PAUSED:
34             wm_fsm_trigger_fault(ctx, WM_FAULT_DOOR_OPEN);
35             return true;
36         case WM_STATE_ERROR:
37             wm_fsm_clear_fault(ctx);
38             return true;
39         default:
40             return false;
41     }
42 }
```



```
43
44 bool wm_single_button_apply(wm_context_t *ctx, hal_gesture_t gesture) {
45     if (ctx == NULL) {
46         return false;
47     }
48
49     switch (gesture) {
50         case HAL_GESTURE_CLICK:
51             return apply_click(ctx);
52         case HAL_GESTURE_DOUBLE:
53             return apply_double(ctx);
54         case HAL_GESTURE_HOLD1:
55             return wm_fsm_dispatch_event(ctx, WM_EVT_BTN_STOP);
56         case HAL_GESTURE_HOLD2: {
57             bool changed = wm_fsm_dispatch_event(ctx, WM_EVT_BTN_STOP);
58             if (ctx->state == WM_STATE_ERROR) {
59                 /* STOP is not defined to leave ERROR; clear the fault so a long hold
always recovers the panel */
60                 wm_fsm_clear_fault(ctx);
61                 changed = true;
62             }
63             return changed;
64         }
65         case HAL_GESTURE_NONE:
66         default:
67             return false;
68     }
69 }
70
71 bool wm_ext_run_pause_apply(wm_context_t *ctx) {
72     if (ctx == NULL || ctx->state == WM_STATE_STANDBY || ctx->state ==
73     WM_STATE_COLLECTING) {
74         return false;
75     }
76     return apply_click(ctx);
77 }
78
79 bool wm_ext_stop_apply(wm_context_t *ctx) {
80     if (ctx == NULL) {
81         return false;
82     }
83     if (ctx->state == WM_STATE_ERROR) {
84         wm_fsm_clear_fault(ctx);
```

```
84     return true;
85 }
86 return wm_fsm_dispatch_event(ctx, WM_EVT_BTN_STOP);
87 }
```

Listing 18: wm_two_button.c

```
1  /**
2   * @file wm_two_button.c
3   * @brief K0/K1 front panel logic with coin staging
4   */
5
6  #include <stddef.h>
7  #include "wm_two_button.h"
8  #include "wm_single_button.h"
9
10 static bool is_idle(const wm_context_t *ctx) {
11     return ctx->state == WM_STATE_STANDBY || ctx->state == WM_STATE_COLLECTING;
12 }
13
14 void wm_staging_init(wm_coin_staging_t *s) {
15     if (s != NULL) {
16         s->pending_cents = 0U;
17     }
18 }
19
20 void wm_staging_sync(wm_coin_staging_t *s, const wm_context_t *ctx) {
21     if (s != NULL && ctx != NULL && !is_idle(ctx)) {
22         s->pending_cents = 0U;
23     }
24 }
25
26 static bool stage_coin(wm_coin_staging_t *s, const wm_context_t *ctx, uint32_t cents)
27 {
28     if (!is_idle(ctx)) {
29         return false;
30     }
31     uint32_t room = WM_STAGING_MAX_CENTS - s->pending_cents;
32     s->pending_cents += (cents > room) ? room : cents;
33     return true;
34 }
35
36 static bool confirm(wm_context_t *ctx, wm_coin_staging_t *s, wm_notice_t *notice,
```



```
uint32_t *value) {
36     if (!is_idle(ctx)) {
37         return false;
38     }
39
40     uint32_t pending = s->pending_cents;
41     s->pending_cents = 0U;
42
43     if (pending >= WM_COIN_THRESHOLD_CENTS) {
44         /* The FSM sees exactly the fare; the excess is reported as returned change */
45         (void)wm_fsm_dispatch_event(ctx, WM_EVT_COIN_50);
46         if (notice != NULL) {
47             *notice = WM_NOTICE_READY;
48         }
49         if (value != NULL) {
50             *value = pending - WM_COIN_THRESHOLD_CENTS;
51         }
52     } else {
53         if (notice != NULL) {
54             *notice = WM_NOTICE_NOT_ENOUGH;
55         }
56         if (value != NULL) {
57             *value = pending;
58         }
59     }
60     return true;
61 }
62
63 bool wm_k1_apply(wm_context_t *ctx, wm_coin_staging_t *s, hal_gesture_t gesture,
64                 wm_notice_t *notice, uint32_t *value) {
65     if (ctx == NULL || s == NULL) {
66         return false;
67     }
68     wm_staging_sync(s, ctx);
69
70     if (is_idle(ctx)) {
71         switch (gesture) {
72             case HAL_GESTURE_CLICK:
73                 return stage_coin(s, ctx, 10U);
74             case HAL_GESTURE_DOUBLE:
75                 return stage_coin(s, ctx, 20U);
76             case HAL_GESTURE_HOLD1_END:
77                 return stage_coin(s, ctx, 50U);
```

```
78         case HAL_GESTURE_HOLD2:
79             return confirm(ctx, s, notice, value);
80         default:
81             return false;
82     }
83 }
84 return wm_single_button_apply(ctx, gesture);
85 }
86
87 bool wm_k0_apply(wm_context_t *ctx, wm_coin_staging_t *s, hal_gesture_t gesture) {
88     if (ctx == NULL || s == NULL) {
89         return false;
90     }
91     wm_staging_sync(s, ctx);
92
93     if (is_idle(ctx)) {
94         switch (gesture) {
95             case HAL_GESTURE_CLICK:
96                 return stage_coin(s, ctx, 10U);
97             case HAL_GESTURE_DOUBLE:
98             case HAL_GESTURE_HOLD1:
99                 return stage_coin(s, ctx, 20U);
100             default:
101                 return false;
102         }
103     }
104     return (gesture == HAL_GESTURE_CLICK) ? wm_ext_run_pause_apply(ctx) : false;
105 }
```

A.3 Board firmware

Listing 19: main_h750.c

```
1 /**
2  * @file main_h750.c
3  * @brief Washing Machine Control Unit on the WeAct STM32H750VBT6 board
4  * @details Input : onboard key K1 (PC13) decoded into click / double-click / hold
5  *                gestures,
6  *                plus optional external RUN/PAUSE (PA0) and STOP (PA1) push-buttons
7  *                Output: dashboard on the 0.96" ST7735 LCD + onboard LED PE3 (mirrors
8  *                RLED/BLED)
```



```
7  *           The FSM, debouncer, blinker and gesture decoder are the same portable
   code that is
8  *           unit-tested on the host; only the board drivers in this folder are
   hardware specific.
9  */
10
11 #include <stdbool.h>
12 #include <stdint.h>
13 #include <string.h>
14 #include "board_h750.h"
15 #include "washing_machine_fsm.h"
16 #include "hal_button_engine.h"
17 #include "hal_button_gesture.h"
18 #include "hal_led_blinker.h"
19 #include "wm_single_button.h"
20 #include "wm_two_button.h"
21 #include "ui_render.h"
22 #include "ui_text.h"
23
24 #define HOLD_UI_STEP_MS   (50U)
25
26 /* Demo build: a "30 minute" cycle is shortened to 30 s so it can be watched live.
27  * Build with FULL_CYCLE=1 (make h750 FULL_CYCLE=1) for the real 1800 s cycle. */
28 #ifndef WM_DEMO_CYCLE_SEC
29 #define WM_DEMO_CYCLE_SEC   (WM_CYCLE_DURATION_TEST_SEC)
30 #endif
31
32 static wm_context_t g_fsm;
33 static hal_button_t g_key;
34 static hal_button_t g_ext_run;
35 static hal_button_t g_ext_stop;
36 static hal_gesture_ctx_t g_gesture;           /* K1 (PC13) */
37 static hal_gesture_ctx_t g_k0_gesture;       /* K0 (external, PA0) */
38 static wm_coin_staging_t g_staging;
39
40 /* Transient message ("READY! CHANGE 050C" / "NOT ENOUGH! REFUND 030C") */
41 #define NOTICE_MS   (2500U)
42 static wm_notice_t g_notice = WM_NOTICE_NONE;
43 static uint32_t g_notice_value = 0U;
44 static uint32_t g_notice_until_ms = 0U;
45 static hal_led_blinker_t g_rled;
46 static hal_led_blinker_t g_bled;
47
```



```
48  /* Output snapshot mirrored on the LCD (relays have no pins on this board) */
49  static hal_motor_state_t g_motor = HAL_MOTOR_OFF;
50  static bool g_pump = false;
51  static bool g_lock = false;
52
53  static void cb_set_rled(hal_led_state_t s) { hal_led_blinker_set_mode(&g_rled, s); }
54  static void cb_set_bled(hal_led_state_t s) { hal_led_blinker_set_mode(&g_bled, s); }
55  static void cb_set_motor(hal_motor_state_t m) { g_motor = m; }
56  static void cb_set_valve(bool open) { (void)open; }
57  static void cb_set_pump(bool on) { g_pump = on; }
58  static void cb_set_lock(bool locked) { g_lock = locked; }
59  static void set_notice(wm_notice_t type, uint32_t value);
60
61  static void cb_return_coins(uint32_t cents) {
62      set_notice(WM_NOTICE_REFUND, cents);
63  }
64
65  static void cb_cycle_complete(void) {
66      g_motor = HAL_MOTOR_OFF;
67      g_pump = false;
68      g_lock = false;
69  }
70
71  /* ----- UART console: status log + keyboard control (115200 8N1 on PA9/PA10) ----- */
72  static const char k_help[] =
73      "K1 keys: c=click(+10c) d=double(+20c) p=hold+release(+50c) o=hold
74      1.2s(OK/confirm)\r\n"
75      "          a=K0 click(+10c) b=K0 hold(+20c) s=STOP (twice) f=force stop
76      ?=help\r\n";
77
78  static void uart_print_uint(uint32_t v) {
79      char tmp[11];
80      unsigned n = 0U;
81      do {
82          tmp[n++] = (char)('0' + (v % 10U));
83          v /= 10U;
84      } while (v != 0U);
85      char one[2] = {0, 0};
86      while (n > 0U) {
87          one[0] = tmp[--n];
88          board_uart_puts(one);
89      }
90  }
```



```
89
90 static bool log_changed(const ui_view_t *a, const ui_view_t *b) {
91     return a->state != b->state || a->phase != b->phase || a->remaining_sec !=
92         b->remaining_sec ||
93         a->balance_cents != b->balance_cents || a->fault_flags != b->fault_flags ||
94         a->motor != b->motor || a->pump != b->pump || a->lock != b->lock;
95 }
96
97 static void log_status(const ui_view_t *v) {
98     char line[128];
99     board_uart_puts("[");
100     uart_print_uint(board_millis() / 1000U);
101     board_uart_puts("s] ");
102     (void)ui_status_line(line, sizeof(line), v);
103     board_uart_puts(line);
104     board_uart_puts("\r\n");
105 }
106
107 static void set_notice(wm_notice_t type, uint32_t value) {
108     g_notice = type;
109     g_notice_value = value;
110     g_notice_until_ms = board_millis() + NOTICE_MS;
111     board_uart_puts(type == WM_NOTICE_READY ? "NOTICE: READY, change returned " :
112         type == WM_NOTICE_REFUND ? "NOTICE: CANCELLED, deposit returned "
113         :
114         "NOTICE: NOT ENOUGH MONEY, refund ");
115     uart_print_uint(value);
116     board_uart_puts("c\r\n");
117 }
118
119 static void k1_event(hal_gesture_t g) {
120     wm_notice_t n = WM_NOTICE_NONE;
121     uint32_t value = 0U;
122     (void)wm_k1_apply(&g_fsm, &g_staging, g, &n, &value);
123     if (n != WM_NOTICE_NONE) {
124         set_notice(n, value);
125     }
126 }
127
128 static void handle_uart_key(int c) {
129     switch (c) {
130         case 'a': board_uart_puts("> K0 click\r\n"); (void)wm_k0_apply(&g_fsm,
131             &g_staging, HAL_GESTURE_CLICK); break;
```

```
129     case 'b': board_uart_puts("> K0 hold\r\n");    (void)wm_k0_apply(&g_fsm,
130     &g_staging, HAL_GESTURE_HOLD1); break;
131     case 'c': board_uart_puts("> K1 click\r\n");    k1_event(HAL_GESTURE_CLICK);
132     break;
133     case 'p': board_uart_puts("> K1 hold+release\r\n");
134     k1_event(HAL_GESTURE_HOLD1_END); break;
135     case 'd': board_uart_puts("> K1 double-click\r\n");
136     k1_event(HAL_GESTURE_DOUBLE); break;
137     case 'o': board_uart_puts("> K1 hold 1.2 s (confirm)\r\n");
138     k1_event(HAL_GESTURE_HOLD2); break;
139     case 's': board_uart_puts("> STOP\r\n");
140     (void)wm_ext_stop_apply(&g_fsm); break;
141     case 'f':
142     board_uart_puts("> force stop\r\n");
143     (void)wm_single_button_apply(&g_fsm, HAL_GESTURE_HOLD1);
144     (void)wm_single_button_apply(&g_fsm, HAL_GESTURE_HOLD2);
145     break;
146     case '?':
147     case 'h': board_uart_puts(k_help); break;
148     default: break;
149 }
150 }
151
152 static void build_view(ui_view_t *v) {
153     v->state = g_fsm.state;
154     v->phase = wm_fsm_get_cycle_phase(&g_fsm);
155     wm_staging_sync(&g_staging, &g_fsm);
156     v->balance_cents = (g_fsm.state == WM_STATE_STANDBY || g_fsm.state ==
157     WM_STATE_COLLECTING) ? g_staging.pending_cents : g_fsm.coin_balance_cents;
158     v->remaining_sec = g_fsm.remaining_cycle_sec;
159     v->total_sec = g_fsm.cycle_duration_setting;
160     v->fault_flags = g_fsm.active_error_flags;
161     v->rled = hal_led_blinker_get_output(&g_rled);
162     v->bled = hal_led_blinker_get_output(&g_bled);
163     v->motor = g_motor;
164     v->pump = g_pump;
165     v->lock = g_lock;
166     v->reset_cause = board_reset_cause();
167     v->bl_mode = 0U;
168     if (g_notice != WM_NOTICE_NONE && (int32_t)(board_millis() - g_notice_until_ms)
169     >= 0) {
170         g_notice = WM_NOTICE_NONE;
171     }
172 }
```



```
164     v->notice = (uint32_t)g_notice;
165     v->notice_value = g_notice_value;
166     v->hold_ms = (hal_gesture_hold_ms(&g_gesture) / HOLD_UI_STEP_MS) *
        HOLD_UI_STEP_MS;
167 }
168
169 /** @brief Deterministic 1 ms housekeeping */
170 static void process_1ms(uint32_t *sec_acc_ms) {
171     wm_fsm_tick_1ms(&g_fsm);
172     hal_led_blinker_tick_ms(&g_rled, 1U);
173     hal_led_blinker_tick_ms(&g_bled, 1U);
174
175     hal_button_update(&g_key, board_key_pressed(), 1U);
176     hal_gesture_t g = hal_gesture_update(&g_gesture, hal_button_is_pressed(&g_key),
        1U);
177     if (g != HAL_GESTURE_NONE) {
178         k1_event(g);
179     }
180
181     /* K0 = external button on PA0 (to GND): coin button in STANDBY, RUN/PAUSE
        otherwise */
182     hal_button_update(&g_ext_run, board_ext_run_pressed(), 1U);
183     hal_gesture_t g0 = hal_gesture_update(&g_k0_gesture,
        hal_button_is_pressed(&g_ext_run), 1U);
184     if (g0 != HAL_GESTURE_NONE) {
185         (void)wm_k0_apply(&g_fsm, &g_staging, g0);
186     }
187
188     hal_button_update(&g_ext_stop, board_ext_stop_pressed(), 1U);
189     if (hal_button_was_pressed(&g_ext_stop)) {
190         (void)wm_ext_stop_apply(&g_fsm);
191     }
192
193     (*sec_acc_ms)++;
194     if (*sec_acc_ms >= 1000U) {
195         *sec_acc_ms = 0U;
196         wm_fsm_tick_1s(&g_fsm);
197     }
198 }
199
200 int main(void) {
201     board_init();
```



```
202 board_led_set(true);          /* boot stage 1: clocks + GPIO OK -> LED on during
LCD bring-up */
203
204 hal_led_blinker_init(&g_rled, true);
205 hal_led_blinker_init(&g_bled, true);
206 hal_button_init(&g_key, false); /* K1 is active HIGH */
207 hal_button_init(&g_ext_run, true); /* external buttons are active LOW */
208 hal_button_init(&g_ext_stop, true);
209 hal_gesture_init(&g_gesture);
210 hal_gesture_init(&g_k0_gesture);
211 wm_staging_init(&g_staging);
212
213 hal_output_callbacks_t cbs = {
214     .set_rled = cb_set_rled,
215     .set_bled = cb_set_bled,
216     .set_motor = cb_set_motor,
217     .set_water_valve = cb_set_valve,
218     .set_drain_pump = cb_set_pump,
219     .set_door_lock = cb_set_lock,
220     .on_cycle_complete = cb_cycle_complete,
221     .return_coins = cb_return_coins
222 };
223 wm_fsm_init(&g_fsm, &cbs);
224 g_fsm.cycle_duration_setting = WM_DEMO_CYCLE_SEC;
225
226 board_uart_init();
227 board_uart_puts("\r\n=== Washing Machine Control Unit - WeAct STM32H750 ===\r\n");
228 board_uart_puts("Reset cause: ");
229 board_uart_puts(board_reset_cause());
230 board_uart_puts("\r\n");
231 board_uart_puts(k_help);
232
233 lcd_init();
234 board_led_set(false);
235
236 ui_view_t shown;
237 ui_view_t now;
238 memset(&shown, 0, sizeof(shown));
239 shown.state = (wm_state_t)0xFF; /* force first draw */
240
241 ui_view_t logged;
242 memset(&logged, 0, sizeof(logged));
243 logged.state = (wm_state_t)0xFF;
```



```
244
245     uint32_t processed = board_millis();
246     uint32_t sec_acc_ms = 0U;
247 #ifdef BL_EXPERIMENT
248     uint32_t bl_mode = 1U;
249     uint32_t bl_since = board_millis();
250     lcd_backlight_mode(bl_mode);
251 #endif
252
253     for (;;) {
254         uint32_t t = board_millis();
255         while (processed != t) {
256             processed++;
257             process_1ms(&sec_acc_ms);
258         }
259
260         build_view(&now);
261 #ifdef BL_EXPERIMENT
262         if ((t - bl_since) >= 4000U) {
263             bl_since = t;
264             bl_mode = (bl_mode % 4U) + 1U;
265             lcd_backlight_mode(bl_mode);
266         }
267         now.bl_mode = bl_mode;
268 #endif
269         int key;
270         while ((key = board_uart_getc()) >= 0) {
271             handle_uart_key(key);
272         }
273         build_view(&now);
274         if (log_changed(&now, &logged)) {
275             log_status(&now);
276             logged = now;
277         }
278         board_led_set(now.rled || now.bled);
279         if (!ui_view_equal(&now, &shown)) {
280             ui_render(&now);
281             lcd_flush();
282             shown = now;
283         }
284     }
285 }
```



Listing 20: board_h750.c

```
1  /**
2   * @file board_h750.c
3   * @brief Clock tree, GPIO and SysTick for the WeAct STM32H750VBT6 board
4   * @details SYSCLK 400 MHz (VOS1, LDO supply) from the 25 MHz HSE crystal. If the
5   *         crystal does not
6   *         start, falls back to HSI 64 MHz with a PLL setting that still yields 400
7   *         MHz, so the
8   *         firmware never hangs at boot and all timing constants stay valid.
9   */
10 #include "stm32h7xx.h"
11 #include "board_h750.h"
12
13 #define SYSCLK_HZ      (400000000U)
14 #define HSE_TIMEOUT_LOOP (2000000U)
15
16 uint32_t SystemCoreClock = SYSCLK_HZ;
17
18 static volatile uint32_t g_millis = 0U;
19 static const char *g_reset_cause = "?";
20
21 const char *board_reset_cause(void) {
22     return g_reset_cause;
23 }
24
25 void SysTick_Handler(void) {
26     g_millis++;
27 }
28
29 uint32_t board_millis(void) {
30     return g_millis;
31 }
32
33 /* Wall-clock wait independent of SysTick/interrupts: DWT cycle counter at SYSCLK_HZ
34 */
35 static void dwt_wait_ms(uint32_t ms) {
36     CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
37     DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
38     uint32_t start = DWT->CYCCNT;
39     uint32_t cycles = ms * (SYSCLK_HZ / 1000U);
40     while ((DWT->CYCCNT - start) < cycles) {
41     }
```



```
40 }
41
42 void board_fatal(uint32_t code) {
43     RCC->AHB4ENR |= RCC_AHB4ENR_GPIOEEN;
44     (void)RCC->AHB4ENR;
45     GPIOE->MODER = (GPIOE->MODER & ~(3U << (3U * 2U))) | (1U << (3U * 2U));
46     for (;;) {
47         for (uint32_t k = 0U; k < code; k++) {
48             GPIOE->BSRR = (1U << 3U);
49             dwt_wait_ms(500U);
50             GPIOE->BSRR = (1U << (3U + 16U));
51             dwt_wait_ms(500U);
52         }
53         dwt_wait_ms(3000U);
54     }
55 }
56
57 void HardFault_Handler(void) {
58     board_fatal(BOARD_ERR_HARDFAULT);
59 }
60
61 void board_delay_ms(uint32_t ms) {
62     uint32_t start = g_millis;
63     uint32_t guard = ms * 4000000U; /* >> loop speed of a 400 MHz M7; only trips if
64                                     SysTick is dead */
65     while ((g_millis - start) < ms) {
66         if (guard-- == 0U) {
67             board_fatal(BOARD_ERR_SYSTICK);
68         }
69     }
70 }
71
72 static bool start_hse(void) {
73     RCC->CR |= RCC_CR_HSEON;
74     for (uint32_t i = 0U; i < HSE_TIMEOUT_LOOP; i++) {
75         if ((RCC->CR & RCC_CR_HSERDY) != 0U) {
76             return true;
77         }
78     }
79     RCC->CR &= ~RCC_CR_HSEON;
80     return false;
81 }
```



```
82 void board_system_init(void) {
83     /* 0. Start from a known clock state. After a USB-DFU "leave" the ROM bootloader
84         jumps here
85         * without a clean reset (PLL1 may already be running, which makes PLL writes
86         ineffective). */
87     RCC->CR |= RCC_CR_HSION;
88     while ((RCC->CR & RCC_CR_HSIRDY) == 0U) {
89     }
90     RCC->CFGR &= ~RCC_CFGR_SW;
91     while ((RCC->CFGR & RCC_CFGR_SWS) != 0U) {
92     }
93     RCC->CR &= ~RCC_CR_PLL1ON;
94     while ((RCC->CR & RCC_CR_PLL1RDY) != 0U) {
95     }
96     __disable_irq();
97     SCB_DisableDCache();
98     SCB_DisableICache();
99
100    /* 1. Supply: LDO, then voltage scale 1 (required for 400 MHz) */
101    PWR->CR3 = (PWR->CR3 | PWR_CR3_LDOEN) & ~PWR_CR3_BYPASS;
102    while ((PWR->CSR1 & PWR_CSR1_ACTIVOSRDY) == 0U) {
103    }
104    PWR->D3CR |= PWR_D3CR_VOS;
105    while ((PWR->D3CR & PWR_D3CR_VOSRDY) == 0U) {
106    }
107
108    /* 2. PLL1: HSE 25/5 = 5 MHz x160 = 800 MHz VCO /2 = 400 MHz (HSI: 64/16 = 4 MHz
109        x200) */
110    bool hse_ok = start_hse();
111    uint32_t divm = hse_ok ? 5U : 16U;
112    uint32_t divn = hse_ok ? 160U : 200U;
113    uint32_t src = hse_ok ? 2U : 0U;
114    RCC->PLLCKSELR = (divm << 4) | src;
115    RCC->PLLCFGR = (2U << 2) | /* PLL1RGE: 4..8 MHz input */
116    RCC_PLLCFGR_DIVP1EN; /* wide VCO, no fractional */
117    RCC->PLL1DIVR = (divn - 1U) | ((2U - 1U) << 9) | ((8U - 1U) << 16) | ((2U - 1U)
118    << 24);
119    RCC->CR |= RCC_CR_PLL1ON;
120    while ((RCC->CR & RCC_CR_PLL1RDY) == 0U) {
121    }
122
123    /* 3. Bus prescalers: CPU 400, AHB/AXI 200, all APB 100 MHz */
124    RCC->D1CFGR = (8U << 0) | (4U << 4); /* HPRE /2, D1PPRE /2, D1CPRE /1 */
```

```
121 RCC->D2CFGR = (4U << 4) | (4U << 8);          /* D2PPRE1 /2, D2PPRE2 /2 */
122 RCC->D3CFGR = (4U << 4);                        /* D3PPRE /2 */
123
124 /* 4. Flash wait states for 200 MHz AXI @ VOS1, then switch to PLL1 */
125 FLASH->ACR = (2U << FLASH_ACR_LATENCY_Pos) | (2U << FLASH_ACR_WRHIGHFREQ_Pos);
126 while ((FLASH->ACR & FLASH_ACR_LATENCY) != (2U << FLASH_ACR_LATENCY_Pos)) {
127 }
128 RCC->CFGR = (RCC->CFGR & ~RCC_CFGR_SW) | (3U << RCC_CFGR_SW_Pos);
129 while ((RCC->CFGR & RCC_CFGR_SWS) != (3U << RCC_CFGR_SWS_Pos)) {
130 }
131
132 SCB->VTOR = FLASH_BANK1_BASE;
133 SCB_EnableICache();
134 SCB_EnableDCache();
135 __enable_irq();
136 }
137
138 void board_init(void) {
139 {
140     uint32_t rsr = RCC->RSR;
141     if ((rsr & RCC_RSR_PORRSTF) != 0U)          { g_reset_cause = "POR"; }
142     else if ((rsr & RCC_RSR_BORRSTF) != 0U)      { g_reset_cause = "BOR"; }
143     else if ((rsr & RCC_RSR_IWDG1RSTF) != 0U)    { g_reset_cause = "IWDG"; }
144     else if ((rsr & RCC_RSR_WWDG1RSTF) != 0U)    { g_reset_cause = "WWDG"; }
145     else if ((rsr & RCC_RSR_SFTRSTF) != 0U)     { g_reset_cause = "SFT"; }
146     else if ((rsr & RCC_RSR_PINRSTF) != 0U)     { g_reset_cause = "PIN"; }
147     RCC->RSR |= RCC_RSR_RMVF;
148 }
149
150 RCC->AHB4ENR |= RCC_AHB4ENR_GPIOAEN | RCC_AHB4ENR_GPIOCEN | RCC_AHB4ENR_GPIOEEN;
151 (void)RCC->AHB4ENR; /* read-back: guarantee clock is running before first access
152 */
153
154 /* LED PE3: push-pull output, off */
155 GPIOE->BSRR = (1U << (3U + 16U));
156 GPIOE->MODER = (GPIOE->MODER & ~(3U << (3U * 2U))) | (1U << (3U * 2U));
157
158 /* KEY PC13: input with pull-down (K1 pulls the pin to 3.3 V when pressed) */
159 GPIOC->MODER &= ~(3U << (13U * 2U));
160 GPIOC->PUPDR = (GPIOC->PUPDR & ~(3U << (13U * 2U))) | (2U << (13U * 2U));
161
162 /* External buttons PA0 / PA1: inputs with pull-up (pressed = LOW) */
163 GPIOA->MODER &= ~(3U << (0U * 2U)) | (3U << (1U * 2U));
```



```
163     GPIOA->PUPDR = (GPIOA->PUPDR & ~((3U << (0U * 2U)) | (3U << (1U * 2U)))) |
164         (1U << (0U * 2U)) | (1U << (1U * 2U));
165
166     (void)SysTick_Config(SYSCLK_HZ / 1000U);
167 }
168
169 void board_led_set(bool on) {
170     GPIOE->BSRR = on ? (1U << 3U) : (1U << (3U + 16U));
171 }
172
173 bool board_key_pressed(void) {
174     return (GPIOC->IDR & (1U << 13U)) != 0U;
175 }
176
177 bool board_ext_run_pressed(void) {
178     return (GPIOA->IDR & (1U << 0U)) == 0U;
179 }
180
181 bool board_ext_stop_pressed(void) {
182     return (GPIOA->IDR & (1U << 1U)) == 0U;
183 }
184
185 #define UART_BAUD (115200U)
186 #define UART_PCLK (100000000U) /* USART1 kernel clock = APB2 (default selection) */
187
188 void board_uart_init(void) {
189     RCC->AHB4ENR |= RCC_AHB4ENR_GPIOAEN;
190     RCC->APB2ENR |= RCC_APB2ENR_USART1EN;
191     (void)RCC->APB2ENR;
192
193     /* PA9 = TX, PA10 = RX: alternate function 7, internal pull-up on RX */
194     GPIOA->MODER = (GPIOA->MODER & ~((3U << (9U * 2U)) | (3U << (10U * 2U)))) |
195         (2U << (9U * 2U)) | (2U << (10U * 2U));
196     GPIOA->AFR[1] = (GPIOA->AFR[1] & ~((0xFU << 4) | (0xFU << 8))) | (7U << 4) | (7U
197 << 8);
198     GPIOA->PUPDR = (GPIOA->PUPDR & ~(3U << (10U * 2U))) | (1U << (10U * 2U));
199     GPIOA->OSPEEDR |= (3U << (9U * 2U));
200
201     USART1->CR1 = 0U;
202     USART1->BRR = UART_PCLK / UART_BAUD;
203     USART1->CR1 = USART_CR1_TE | USART_CR1_RE | USART_CR1_UE;
204 }
```

```
205 static void uart_putc(char c) {
206     uint32_t guard = 2000000U;
207     while ((USART1->ISR & USART_ISR_TXE_TXFNF) == 0U) {
208         if (guard-- == 0U) {
209             return; /* never block the machine on a missing terminal */
210         }
211     }
212     USART1->TDR = (uint32_t)(uint8_t)c;
213 }
214
215 void board_uart_puts(const char *s) {
216     while (*s != '\0') {
217         uart_putc(*s++);
218     }
219 }
220
221 int board_uart_getc(void) {
222     if ((USART1->ISR & USART_ISR_ORE) != 0U) {
223         USART1->ICR = USART_ICR_ORECF;
224     }
225     if ((USART1->ISR & USART_ISR_RXNE_RXFNE) != 0U) {
226         return (int)(USART1->RDR & 0xFFU);
227     }
228     return -1;
229 }
```

B Pin configuration

The project does not use STM32CubeMX: peripherals are configured directly through the registers (board_h750.c, lcd_st7735.c) following RM0433^[2]. The resulting configuration is:



Table B.1: Register-level pin configuration

Pin	MODER	PUPDR	AF	Function
PC13	00 input	10 pull-down	–	K1
PA0, PA1	00 input	01 pull-up	–	external K0, STOP
PE3	01 output	–	–	status LED
PE11, PE13	01 output	–	–	LCD CS, DC
PE12, PE14	10 alternate	–	AF5	SPI4 SCK, MOSI
PE10	10 alternate	–	AF1	TIM1_CH2N backlight PWM
PA9, PA10	10 alternate	PA10 pull-up	AF7	USART1 TX, RX

C Full test logs

Listing 21: make test – FSM suite

```

1 =====
2 BTL 2: Washing Machine Control Unit - Verification Suite
3 CQ3053 Embedded Systems - HCMUT
4 =====
5
6 [PASS] TC-01: Sub-threshold Deposit (10¢ + 20¢ stays in COLLECTING, RLED on)
7 [PASS] TC-02: Exact Threshold Deposit (50¢ transitions to READY)
8 [PASS] TC-03: Surplus Deposit Accumulation (Accepts 60¢, 110¢ in READY)
9 [PASS] TC-04: Execution & Zero Refund (70¢ cleared to 0¢, 30-min timer active)
10 [PASS] TC-05: Premature RUN Attempt (Ignored when balance < 50¢)
11 [PASS] TC-06: Normal Pause and Resume (Actuators safely suspended and resumed)
12 [PASS] TC-07: Persistent Timer in Pause (Timer ticked down from 1600s to 1300s while paused)
13 [PASS] TC-08: Pause Timeout Termination (Timer expiring in PAUSED resets to STANDBY)
14 [PASS] TC-09: Single STOP Rejection (Single press does not force stop)
15 [PASS] TC-10: Force Stop on Double Press (2 presses within 300ms forces termination)
16 [PASS] TC-11: Force Stop from Paused State (Double STOP terminates paused machine)
17 [PASS] TC-12: Normal 30-min Cycle Completion (1800s expires naturally to STANDBY)
18 [PASS] TC-13: Fault Interruption and Recovery (Safety shutdown & error reset)
19 [PASS] TC-14: Rapid Alternating PAUSE/RUN Toggling (Stress test on clock & motor)
20 [PASS] TC-15: Coin Rejection During Active Cycle (Coins rejected in RUNNING, PAUSED, ERROR)
21 [PASS] TC-16: Fault in STANDBY and READY States (Consistent error transition & LED signalling)
22 [PASS] TC-17: Complete Lockout During ERROR State (Buttons and coins strictly locked)
23 [PASS] TC-18: Ready State Double STOP Cancellation (User cancel before run)
24 [PASS] TC-19: Multiple Isolated Single STOPS (Spaced presses never falsely force stop)
25 [PASS] TC-20: Null Pointer and API Resilience (Zero segmentation faults, robust error handling)
26 [PASS] TC-21: Consecutive Multi-Cycle Sessions (Flawless back-to-back operations without leakage)
27 [PASS] TC-22: Boundary Double-Stop Timing (Exact 1499ms hit vs 1501ms expiration verified)
28 [PASS] TC-23: Single STOP in READY (Preserves accumulated deposit against accidental touch)
29 [PASS] TC-24: Granular Fault Diagnostics (Multi-sensor bitmask tracking and string reports)
30 [PASS] TC-25: Arithmetic Overflow Resilience (MISRA-C Rule 12.4 wrap-around defense)

```



```
31 [PASS] TC-26: Multi-Phase Wash Profile (Agitate -> Spin/Drain -> Complete verified)
32 [PASS] TC-27: Event Acceptance Query Protocol (Deterministic event filtering across all 5 states)
33 [PASS] TC-28: Cycle Sub-Phase Query & Enum Decoders (Correct phase detection throughout cycle)
34 [PASS] TC-29: Pause Across Phase Boundary (Continuous timer crosses into Spin)
35 [PASS] TC-30: MISRA-C Boundary & Corrupted Enum Resilience (100% defensive branch safety)
36 [PASS] TC-31: COLLECTING State (RLED on until the 50¢ threshold, then READY)
37 [PASS] TC-32: Cancel Returns Deposit (STOP x2 in COLLECTING/READY refunds; RUN still zero-refund)
38 [PASS] TC-33: Fault Preserves Deposit (COLLECTING/READY restored after the fault is cleared)
39 [PASS] TC-34: Fault During Cycle (timer keeps counting, cleared fault resumes in PAUSED, expiry
    respected)
40
41 =====
42 ALL 34 TESTS PASSED SUCCESSFULLY! (100% Specification & Transition Coverage)
43 =====
```

Listing 22: make test_hal – HAL engines

```
1 =====
2 BTL 2: Hardware HAL Engines - Unit Verification
3 CQ3053 Embedded Systems - HCMUT
4 =====
5
6 [PASS] HAL-01: Button Debounce (30ms glitch rejection, overflow defense and edge detection)
7 [PASS] HAL-02: Coin Pulse Validation (Multi-pulse train decoding verified)
8 [PASS] HAL-03: LED Blinker Waveforms (Accurate 1.0 Hz and 2.0 Hz non-blocking timing & zero-drift)
9 [PASS] HAL-04: Actuator Interlock Guard (Strict hardware protection rules verified)
10 [PASS] HAL-05: Coin Pulse FIFO Queue (Burst coin insertion without event drops)
11 [PASS] HAL-06: NULL Pointer Resilience (Zero crash across all HAL API functions)
12
13 =====
14 ALL 6 HARDWARE HAL ENGINE TESTS PASSED!
15 =====
```

Listing 23: make test_gesture – single-button gestures

```
1 === Single-button gesture tests ===
2 [PASS] click is delayed until the double-click gap expires
3 [PASS] short press -> exactly one CLICK
4 [PASS] two short presses -> exactly one DOUBLE (no CLICK)
5 [PASS] hold 0.6 s -> HOLD1 while still pressed
6 [PASS] keep holding to 1.2 s -> HOLD2
7 [PASS] no repeat events while the key stays down
8 [PASS] no CLICK after a hold is released
9 [PASS] release between 0.6 s and 1.2 s -> HOLD1_END
10 [PASS] release after 1.2 s -> no HOLD1_END
11 [PASS] press-release-hold -> HOLD1, pending click dropped
12 [PASS] HOLD1->HOLD2 interval fits inside the FSM STOP double-press window
13 [PASS] init STANDBY
14 [PASS] DOUBLE in STANDBY -> +10c (COLLECTING)
15 [PASS] CLICK in STANDBY -> +50c -> READY
16 [PASS] CLICK in READY -> RUNNING
17 [PASS] CLICK in RUNNING -> PAUSED
18 [PASS] CLICK in PAUSED -> RUNNING (resume)
19 [PASS] DOUBLE while RUNNING -> door-open fault -> ERROR
```



```
20 [PASS] CLICK in ERROR -> cleared, cycle resumes in PAUSED
21 [PASS] STANDBY after clearing / force stop
22 [PASS] restart cycle
23 [PASS] HOLD1 only arms STOP
24 [PASS] HOLD2 completes force STOP -> STANDBY
25 [PASS] fault injected
26 [PASS] HOLD in ERROR clears the fault (deposit/cycle context restored by the FSM)
27 [PASS] ext RUN/PAUSE ignored in STANDBY
28 [PASS] ext RUN/PAUSE: READY -> RUNNING
29 [PASS] ext RUN/PAUSE: RUNNING -> PAUSED
30 [PASS] ext RUN/PAUSE: PAUSED -> RUNNING
31 [PASS] ext STOP: single press only arms
32 [PASS] ext STOP: double press -> STANDBY
33 [PASS] ext STOP clears ERROR
34 Passed: 32, Failed: 0
```

Listing 24: make test_two_button – front panel

```
1 === Two-button front panel tests ===
2 [PASS] K1 click: +10c staged, FSM untouched
3 [PASS] K1 double-click: +20c
4 [PASS] K1 HOLD1 alone changes nothing (still deciding)
5 [PASS] K1 hold+release: +50c, still waits for confirm
6 [PASS] K1 hold to 1.2 s with 80c -> READY, 30c change returned
7 [PASS] FSM balance is exactly the fare
8 [PASS] exactly 50c -> READY, no change
9 [PASS] 30c confirm -> not enough, 30c refunded, stays STANDBY
10 [PASS] confirm with nothing inserted -> not enough
11 [PASS] staged coins are capped
12 [PASS] K0 click +10c, K0 hold +20c
13 [PASS] READY: K1 click -> RUNNING
14 [PASS] RUNNING: K0 click -> PAUSED
15 [PASS] PAUSED: K0 click -> RUNNING
16 [PASS] K0 hold outside STANDBY adds no money
17 [PASS] K1 hold 2 stages while RUNNING -> force STOP
18 [PASS] ERROR clears staged coins
19 [PASS] K0 click in ERROR clears the fault
20 ALL TWO-BUTTON TESTS PASSED
```

Listing 25: make test_ui_text – UART status line

```
1 [PASS] returned length matches
2 [PASS] RUNNING line
3 [PASS] STANDBY shows full cycle time, coin and fault
4 [PASS] truncation is safe and NUL-terminated
5 ALL UI TEXT TESTS PASSED
```